

Arbeitsdokument: Unified-Persistence-Konzept — Queries, Change-Streams, Zeitreihen, Metadaten

Status: Arbeitsdokument — Konzept- und Design-Diskussion, noch kein Implementierungsplan. Hält die Architekturdiskussion vom 18.07.2026 fest (einheitliche Query-Sprache für JPA/Mongo, Diff/Patch-Persistenz, Zeitreihen, Metamodell-Evolution, Fehlertoleranz, Transport). Verifiziert gegen die bestehenden Assets in `fennec.common.models`, `emf.model.metadata`, `emf.util` (`org.eclipse.fennec.sensinact.mapping`), `org.eclipse.emf-compare` und `org.eclipse.emf` (`ecore.change`, `emf.edit`).

1. Anforderungen & Scope

Sieben Anforderungscluster treiben dieses Konzept:

1. **Einheitliche Query-Sprache** über alle Persistenz-Backends (JPA/EclipseLink, MongoDB, künftige), inklusive CRUD. Ein Read-only-Query-Metamodell existiert bereits (`fennec.common.models/org.eclipse.fennec.query.model`).
2. **Zeitreihen-Persistenz** — einzelne Features einer EClass (z. B. `Sensor.temp`) werden als Wertehistorie geführt, neben dem Ist-Zustands-EObject.
3. **Diff/Patch-Verarbeitung** — Diffs erzeugen, persistieren, auf Objekte anwenden, invertieren. Diffs sind selbst EMF-Objekte. EMF Compare ist der Referenzpunkt, seine Laufzeitauglichkeit ist aber begrenzt (siehe §3.3).
4. **Zwei Arten von Daten** — schlichte Ist-Zustands-Persistenz von EObjects (wie heute) und/oder Historie (Changelog / Zeitreihe) pro Feature. Beide Repräsentationen koexistieren: der Sensor hält die *aktuelle* Temperatur, die Serie hält die *Historie*.
5. **Die Historie ist ein konfigurierbares Changelog** — pro Feature: ob getrackt wird, unter welchen Bedingungen (Schwellwerte) und wie lange aufbewahrt wird (Retention).
6. **Gebündelte Änderungen** — ein Speichern im Editor ändert mehrere Features auf einmal, dennoch müssen einzelne Änderungen selektiv rückgängig gemacht / angewendet werden können (vgl. `org.eclipse.emf.edit` Command/CommandStack).
7. **Modell-Metadaten** — Dokumentation, Privacy-Beschreibungen, Genmodels, Codec-Mappings, Maßeinheiten — werden *getrennt* von den EObjects gehalten (`emf.model.metadata`).

Dazu aus der Diskussion:

8. **Verschachtelung** — Containment-Hierarchien müssen in Diffs darstellbar sein.
 9. **Fehlertoleranz** — explizite Policies für fehlende Sequenzen, Out-of-Order- und Duplikat-Zustellungen; Reparaturmechanismen (Get-full / State-Resync, Keyframing).
 10. **Transport** — Diffs müssen zwischen Systemen übertragbar sein, ohne dass der Empfänger die Domänenmodell-Version des Senders vorab kennt.
-

2. Kernprinzipien

Alles Folgende leitet sich aus vier Prinzipien ab. Ist eine Detailentscheidung unklar, entscheiden diese.

P1 — Streams sind die Wahrheit; alles andere ist eine Projektion.

Der Change-Stream (geordnetes Log von Deltas) ist die maßgebliche Aufzeichnung. Das Ist-Zustands-EObject, Indizes, abgeleitete Metadaten (`lastSequence` , `changeDate` , `creationDate`) und künftige Aggregate (min/max/avg) sind *materialisierte Projektionen* über Streams: rekonstruierbar, asynchron gepflegt, eventually consistent. Der Verlust einer Projektion bedeutet Neuindizierung, nie Datenverlust (das Lucene-Modell: Dokumente sind die Wahrheit, der Index ist verzichtbar).

Wo `ChangeRules` einen Stream filtern (§8.1), ist die Invariante toleranzbegrenzt: der Stream ist verlustfrei *relativ zu den geltenden Regeln*, und `fold(stream) ≈ Ist-Zustand` innerhalb der deklarierten Toleranz — exakte Gleichheit, wenn keine Regeln konfiguriert sind.

P2 — Alles ist EMF.

Queries, Patches, Serienpunkte, Keyframes, Metadaten-Snapshots sind EMF-Objekte und laufen durch dieselbe Persistenzmaschinerie („Dogfooding“ — `Query.saveQuery` ist das erste existierende Beispiel).

P3 — Capture geschieht an der Persistenzgrenze; State heilt Events.

Kein systemweiter, permanent aktiver Change-Notification-Bus. Change-Capture ist lokal zum Prozess / zur Transaktion, die den Schreibvorgang ausführt. Zwischen Systemen wird das persistierte Log (Pull/Replay) oder Full-State-Transfer genutzt — nie Fire-and-forget-Events. Eine verlorene Nachricht kostet ein Sample, nicht die Konsistenz, denn der nächste Full State repariert alles (state-basiert, selbstheilend).

P4 — Die stabilen Koordinaten sind IDs, nicht Namen.

Objekte werden über stabile Objekt-IDs adressiert, Features über stabile Feature-IDs, die Interpretation von Werten über einen Kontext-Fingerprint. Namen sind Metadaten und dürfen sich frei ändern.

2.1 Architekturüberblick: drei Ebenen

Alles Folgende ordnet sich in drei Ebenen — alle davon EMF (P2):

```

| Metamodel1-Ebene -----
| Ecore + Aspektmodelle: eorm, Codec-Mapping, Docs, Privacy, Units,
| Tracking-Config (§8) – registriert in der Aspekt-Registry (§3.2)
| → deklariert, WAS persistiert/getrackt wird und WIE
|
| Daten-Ebene -----
| Ist-Zustand (EObject, wie heute) + Change-Streams (ChangeSets /
| Serien-Samples – selbst EMF-Objekte) + Metadaten-Snapshot-Kette (§6)
| → der Ist-Zustand ist die materialisierte Sicht des Streams (P1)
|
| Service-Ebene -----
| PersistenceResource / EntityManager (heute)
| QueryService – Query-Modell → Backend-Interpreter (§3.1, §14)
| PatchService – record / apply / invert / Konfliktprüfung (§5, §13)
| SeriesService – append / Range-Query / Housekeeping (§9, §10)
| FingerprintService + Snapshot-Registry (§6)

```

Backend-Adapter (EclipseLink, Mongo, ...), jeder deklariert, welche Capabilities er nativ bedient

3. Bestehende Assets und ihre Rollen

3.1 Query-Modell

(`fennec.common.models/org.eclipse.fennec.query.model`)

`Query` (Subjects = FeaturePaths, Where-Baum aus `Comparator`n, `sortBy/groupBy/limit/skip/distinct`, `saveQuery`) ist backend-neutral und deckt den Lesefall ab. Es wird zum **Quellmodell einer Interpreter-SPI pro Backend**: ein `QueryProcessor`-Service je Backend übersetzt die EMF-Query in die native Form (JPQL/Criteria für EclipseLink, Filter-Dokumente für Mongo).

Bekannte Lücken, die zu schließen sind:

- **Capability-Deklaration pro Backend.** Nicht jedes Backend kann alles (z. B. `Average` + `groupBy` über tiefe FeaturePaths). Prozessoren deklarieren native Capabilities und schlagen mit Diagnostics fehl, statt stillschweigend im Speicher nachzufiltern.
- **Parameter/Platzhalter** für Prepared Queries (nahezu Pflicht, sobald `saveQuery` genutzt wird).
- **Typisierte Comparator-Werte** — Comparators tragen heute `EString`-Werte; die Konvertierung gehört an eine Stelle (den `ConverterService`), nicht in jeden Backend-Prozessor.
- **Nicht-EObject-Projektionen** — `aliasFeature` deutet es an, der Ergebnistyp ist unspezifiziert.
- **Zeitdimension** — `asOf` (Point-in-time-Reads) und Range-Queries über Serien (§14).

CRUD: Insert und Delete sind trivial (Delete = Query-Selektor). **Update = Query-Selektor + Patch** — sobald das Patch-Modell existiert, braucht die Query-Sprache kein eigenes Update-Vokabular.

3.2 Metadaten-Service (`emf.model.metadata`)

Das `metadata.ecore` hat bereits die Aspekt-Registry-Struktur, die dieses Konzept braucht: `Aspect` / `PackageAspect` / `ClassAspect` / `FeatureAspect` plus `PackageProfile` / `ClassProfile`, geschlüsselt über Ecore-Elemente. Aspekte sind eigenständige Modelle (keine EAnnotations) — richtig so, denn Aspekte haben eigenen Lifecycle, eigene Autoren und eigene Ladbarkeit.

Verallgemeinert wird daraus eine **Aspekt-Registry**: ein OSGi-Service, bei dem Aspektmodelle (orm, Codec-Mapping, Docs, Privacy, Units, Tracking-Config) registriert und unter dem Schlüssel (`nsURI`, `EClass`, `Feature`) abgefragt werden.

Die dortige offene Frage — *Registry vs. Index-Holder* — löst P1: **beides, geschichtet**. Die Registry ist die API (Single Point of Contact als Service-Interface); der Index ist eine materialisierte Sicht dahinter (pro EClass/Objekt: Verweise auf Metadaten-Dateien und auf Zeitreihen), asynchron gepflegt. Er muss **aus den Quellen rekonstruierbar** sein; dann ist er ein Cache, keine Wahrheit, und die Single-Point-of-Failure-Sorge löst sich auf.

3.3 EMF Compare (org.eclipse.emf-compare)

EMF Compare löst *Zwei-Zustands-Vergleich zur Designzeit*: beide Modellversionen vollständig im Speicher, teures Matching (identifizier- oder proximity-basiert), und das `Comparison`-Modell ist **nicht self-contained** — `Match.left/right/origin` und die Diffs referenzieren die *lebenden* EObjects beider Seiten. Serialisierte Comparisons halten Proxy-URLs auf Zustände, die später nicht mehr existieren.

Rolle in diesem Konzept: **optionaler Offline-Generator** (zwei persistierte Snapshots vergleichen → `Comparison` → Patch-Batches konvertieren) und die Referenz für die **Konflikterkennungs**-Logik (§13). Er ist ein Zulieferer, nicht das Fundament.

3.4 EMF-Change-Modell (org.eclipse.emf.ecore.change)

`ChangeRecorder`/`ChangeDescription` zeichnet `FeatureChange`s über Adapter auf, ist ein EMF-Modell, unterstützt `applyAndReverse()`. Aber es referenziert Objekte direkt (nicht über stabile IDs), ist als transientes Undo-Artefakt konzipiert und — entscheidend — **trägt nicht für state-basierten Ingest** (§8.1): wenn ein vollständiger neuer Zustand ankommt (TTN-JSON → Sensor-EObject), gibt es nichts aufzuzeichnen.

Rolle: Design-Inspiration für die Recording-Capture-Quelle, nicht das Persistenzformat.

3.5 EMF Edit (org.eclipse.emf/plugins/org.eclipse.emf.edit)

Command/CommandStack ist das **In-Memory-Undo** innerhalb einer Editing-Domain. Der Patch-Batch ist sein **persistentes Gegenstück**: ein Batch = ein Commit = ein Editor-Save. Die Commit-Grenze ist zugleich die Antwort auf das Event-Rauschen (§8.2).

3.6 sensiNact-Mapping

(emf.util/org.eclipse.fennec.sensinact.mapping)

Die `PersistenceRuleRegistry` in `sensinact-mapping.ecore` enthält bereits, praxiserprobt, genau die Change-/Housekeeping-Regeln, die hier gebraucht werden:

Regel	Semantik
<code>AbsoluteChangeRule(delta)</code>	nur speichern, wenn $ \Delta \geq \text{delta}$ (z. B. „nur Änderungen > 0.5 tracken“)
<code>PercentageChangeRule(percentage)</code>	nur speichern, wenn Änderung $\geq n\%$ des letzten gespeicherten Werts („±10 %“)
<code>CountChangeRule(n)</code>	1 von je n Notifications speichern
<code>TimeThrottleChangeRule(interval, unit)</code>	höchstens einmal pro Intervall speichern
<code>DeletionRule(retention, retentionUnit, maxCount, cleanupInterval)</code>	Bereinigung nach Alter und/oder Obergrenze pro Ressource („letzte 100 Werte behalten“, „3 Tage für Feature A, 5 Tage für Feature B“)

Diese sind konzeptionell richtig, leben aber im falschen Namespace. **Sie sollten in ein neutrales Fennec-Tracking-Modell gehoben werden** (den Serien-/Changelog-Config-Aspekt); das sensiNact-Mapping referenziert sie dann. Dieselben Regeln müssen auf dem JPA-, Mongo- und sensiNact-Pfad gelten.

4. Das Koordinatensystem

Jeder Änderungseintrag trägt sechs Koordinaten:

```
(objectId, featureId, sequence, timestamp, contextFingerprint, delta)
Identität Ort      Ordnung  Domänenzeit Interpretation  Änderung
```

4.1 Stabile Objekt-IDs — der Identitätsvertrag

Voraussetzung für alles andere. Patches, Serien, Keyframes adressieren Objekte über IDs. Die Anforderung folgt der **Stream-Sichtbarkeit**, nicht dem Containment per se: was in einem Stream auftaucht — als Eintragssubjekt oder als Referenzziel in einem Delta-Wert — braucht eine stabile ID, und die Anforderung vererbt sich *nach unten* durch getrackte Containments (Kinder, die in Containment-Deltas referenziert werden, brauchen IDs, auch wenn ihre eigenen Features ungetrackt sind). Das ist ein Framework-Vertrag, nicht länger „pro Backend anders gelöst“ (entscheidet die offene Frage §17.1).

Identität wird pro Containment-Feature im Tracking-Aspekt deklariert (`IdentityStrategy`; Aggregate Roots nutzen `NATURAL` oder `SYNTHETIC`):

- **NATURAL** — das EMF-ID-Attribut oder deklarierte Schlüssel-Features (`matchKeys`, Wiederverwendung der ID-Strategie-Infrastruktur des Metadaten-Modells, §3.2).
Erforderlich für mehrwertige Containments, die per Snapshot-Ingest gefüllt werden (§7.1) — nichts anderes kann frische Deserialisierungen korrelieren.
- **SLOT** — Identität *abgeleitet vom Ort*: `parentId/featureId` (einwertig — immer sicher, der Slot kann nie umsortiert werden) oder `parentId/featureId[index]` (mehrwertig — **nur** dort, wo der Index Domänenidentität ist, z. B. feste Kanalpositionen; umsortierbare Listen dürfen es nicht verwenden). Das ist die Antwort für wertartige Kinder ohne jede ID, wie `GeoLocation{lat, lon}`: die Identität ist „die Position von Sensor 42“ — das Ersetzen der Kindinstanz ist *by design* ununterscheidbar vom Mutieren, und die `lat`-Serie bleibt über Instanzwechsel hinweg kohärent. Slot-IDs sind deterministisch und koordinationsfrei (jeder Knoten leitet dieselbe ID ab — ingest-freundlich). Einschränkungen: keine eingehenden Cross-Referenzen, keine Migration. **Das Umhängen (Re-Parenting) eines slot-identifizierten Kindes ist kein Move, sondern ein Ende und ein Anfang**: die Historie von `42/f3` endet (`DELETE`), die Historie von `43/f3` beginnt (`CREATE` + Werte), transaktionsverknüpft, wenn zwei Streams beteiligt sind — vollständig geloggt, bewusst ohne `MIGRATE`-Paar (eine Herkunftsreferenz würde Instanzidentität zurückschmuggeln). Die Modellierungsfrage lautet daher: *gehört die Historie zum Ort oder zum Ding?* Ort → **SLOT** (Re-Parent = Ende + Anfang); Ding (z. B. ein Batteriepack, das zwischen Geräten getauscht und für seine eigene Verschleißhistorie getrackt wird) → `NATURAL` / `SYNTHETIC`, und Re-Parenting wird zu einer Migration, deren Historie mitwandert (§5.4e).
- **SYNTHETIC** — UUID, bei der Erzeugung geprägt und **mit dem Ist-Zustand persistiert**: die

Korrelation Stream-ID ↔ persistiertes Objekt muss den DB-Roundtrip überleben (für JPA kann das eine Schema-Ergänzung bedeuten — Tracking zu aktivieren ist dann eine Migration). Gut für RecordingCapture-Pfade; **nicht nutzbar** für reinen Snapshot-Ingest mehrwertiger Kinder (eine frische Deserialisierung kann nicht zu einer UUID korreliert werden, die sie nie trug).

- **EMBEDDED** — keine Identität: das Kind ist ein atomarer *Wert* des Eltern-Features (das ganze codierte Objekt reist in `SET / ADD`-Deltas, analog zu `ArrayMode.ATOMIC`). Kein `CREATE / DELETE`, kein Matching, keine ID-Inflation für Wertobjekte (`Money{amount, currency}`). Einschränkungen: nicht von anderswo referenziert, nicht selbst getrackt.

AUTO löst auf: EMF-ID-Attribut vorhanden → **NATURAL**; einwertiges Containmentment → **SLOT**; andernfalls verlangt das Tooling eine explizite Wahl (ein mehrwertiges Containmentment ohne natürlichen Schlüssel ist eine *Entscheidung*, kein Default).

Identitätsregeln (normativ)

Bindend für Capture, Apply, Housekeeping und Validierung (entschieden 18.07.2026, §22 Entscheidung #9):

- **I1 — Stream-Sichtbarkeit erfordert Identität.** Was in einem Stream auftaucht — als Eintragssubjekt oder als Referenzziel in einem Delta — hat eine stabile ID; die Anforderung vererbt sich durch getrackte Containments nach unten. **EMBEDDED**-Kinder sind die einzige Ausnahme: sie erscheinen nie als Subjekte oder Ziele, nur als codierte Werte.
- **I2 — SLOT-Identität ist Ortshistorie.** `parentId/featureId` (einwertig: immer erlaubt) oder `parentId/featureId[index]` (mehrwertig: nur bei deklarierter Positionsemantik — umsortierbare Listen nie). Slot-IDs sind abgeleitet, deterministisch und koordinationsfrei; jeder Knoten berechnet dieselbe ID.
- **I3 — Re-Parenting eines SLOT-Kindes ist ein Ende und ein Anfang.** `DELETE` des alten Slot-Objekts + `CREATE` des neuen mit Wertkopie, verknüpft über `transactionId`, wenn zwei Streams beteiligt sind. Nie ein `MIGRATE`-Paar. Historienkontinuität über Re-Parenting hinweg wird bewusst *nicht geboten* — wird sie gebraucht, ist **SLOT** die falsche Strategie (I5).
- **I4 — SLOT-Objekte nehmen keine eingehenden Cross-Referenzen an.** Ihre Identität löst sich beim Re-Parenting auf, also darf nichts außerhalb des Elternteils auf sie zeigen; die Validierung erzwingt das.
- **I5 — NATURAL / SYNTHETIC-Identität ist Dinghistorie.** Re-Parenting über Aggregate hinweg ist ein Migrationspaar (§5.4e); die Historie folgt dem Objekt.
- **I6 — SYNTHETIC-IDs werden mit dem Ist-Zustand persistiert.** Die Korrelation Stream-ID ↔ Objekt muss den DB-Roundtrip überleben (JPA: ggf. eine Schema-Ergänzung); **SYNTHETIC** ist für Snapshot-Ingest mehrwertiger Kinder unbrauchbar.
- **I7 — Mehrwertiger Snapshot-Ingest erfordert NATURAL-matchKeys.** Ohne sie degradiert Capture zu Replace-all (Log-Churn), und das Tooling warnt.
- **I8 — AUTO-Auflösung.** EMF-ID-Attribut → **NATURAL**; einwertiges Containmentment → **SLOT**; andernfalls wird eine explizite Wahl verlangt.

4.2 Stabile Feature-IDs (die Protobuf-Disziplin)

Das Tracking-Aspektmodell weist jedem getrackten Feature eine **unveränderliche ID** zu (das Protobuf-Feldnummern-Prinzip). Log-Einträge speichern die Feature-ID, nie den Namen. Folgen:

- Ein Rename `temp` → `temperatur` ändert *nur Metadaten* — null Log-Einträge. Das löst das Fan-out-Problem: bei 1 Mio. Instanzen einer EClass würde ein naives Design („Modell-Events in Objekt-Logs injizieren“) das Rename-Event 1 Mio. Mal schreiben. Mit Feature-IDs wird es **null** Mal geschrieben (einmal in die Metadaten-Snapshot-Kette, §6.4).
- Disziplinregeln (wo möglich durch Tooling erzwungen): eine ID ist an die *Semantik* eines Features gebunden, nicht an seinen Namen; IDs werden nie wiederverwendet; Renames ändern die ID nie.
- Disziplin lässt sich über Repo-Grenzen und Jahre hinweg nicht erzwingen — genau deshalb existiert darunter der Kontext-Fingerprint (§6) als Sicherheitsnetz.

Das natürliche Zuhause der ID-Zuweisung ist der eorm-/Metadaten-Aspekt (wo Feature-level Mapping-Konfiguration bereits lebt).

4.3 Sequence vs. Timestamp — zwei Zeiten

Die Sensor-Realität kennt **Event Time** (Geräte-Zeitstempel; kann verspätet und außer der Reihe ankommen — LoRaWAN-Retransmits) und **Commit Time** (wann der Eintrag im Log landet). Wenn „Version = Timestamp“ gilt und ein spätes Paket ankommt, sind Versionen nicht mehr append-monoton. Daher:

- `sequence` — **pro Stream monotoner long**, maßgebliche Ordnung. Genutzt für Gap-Erkennung, Replay, Undo, Idempotenz.
- `timestamp` — Domänenattribut (Event Time). Genutzt für Range-Queries und Zeitreihensemantik.

Im einfachen Fall fallen beide zusammen (die Sequence darf buchstäblich Epoch-Millis sein, wenn der Producer der einzige Schreiber ist); das Modell trägt beide Felder, damit der erste verspätete Uplink die Ordnung nicht korrumpiert.

Streams sind Single-Master (normativ, §22 Entscheidung #6). Die lückenlose Sequence verlangt genau einen Sequencer pro Stream: der **kanonische Log-Store vergibt Sequences beim Schreiben**. Producer (z. B. zwei Gateways, die denselben Sensor hören) sind dem Sequencing *vorgelagert* — sie deduplizieren über `ChangeSet.id`, und Gap-Erkennung existiert nur vom Store in Richtung Replikate. Replikate sind read/apply-only. Multi-Master-Schreibzugriffe auf einen Stream sind *per Konstruktion* unmöglich — das ist eine deklarierte Einschränkung, kein Versehen; Knoten-übergreifende Synchronisation ist state-basiert oder replay-basiert (P3, §12). *Zeitreihe und Changelog sind technisch dasselbe* — die Serie ist ein Changelog, dessen Version zufällig zeitkorreliert ist; die Vereinheitlichung ist Absicht (§9).

4.4 Kontext-Fingerprint

Siehe §6. Ein 8-Byte-Wert, inhaltsabgeleitet, der den Interpretationskontext (Modell + log-relevante Metadaten) identifiziert, unter dem der Eintrag geschrieben wurde.

5. Änderungseintrag und Patch-Metamodell

5.1 Eintragsstruktur (Skizze, Namen TBD)

```
ChangeSet (= Batch)                                // ein Commit / ein Editor-Save / eine Ingest-
Nachricht
  id          : UUID
  streamId    : Objekt-/Aggregat-Stream, zu dem dieser Batch gehört (§5.4)
  sequence    : long                               // pro Stream, lückenlos
  timestamp   : long                               // Event Time (Epoch-Millis)
  commitTime  : long                               // Ingest-/Commit-Zeit
  contextFp   : bytes[8]                          // zusammengesetzter Kontext-Fingerprint (§6.4,
$22 Entscheidung #2)
  author/cause : Provenienz (User, Gerät, Import-Job, ...)
  entries     : ChangeEntry[*]                    // geordnet!

ChangeEntry
  objectId    : stabile Objekt-ID
  eClassRef   : über contextFp + Klassen-ID (nicht über den Namen)
  kind        : CREATE | SET | UNSET | ADD | REMOVE | MOVE | PUT | REMOVE_KEY
               | SET_AT | RESHAPE | DELETE | KEYFRAME
  featureId   : stabile Feature-ID                // fehlt bei CREATE/DELETE/KEYFRAME
  address     : Sub-Feature-Adresse (§5.4h/i) – index:int (geordnete Listen),
               key:codiertes Literal|objectId (Maps), coords:int[] (Arrays)
  toIndex     : int                               // nur MOVE
  valueOld    : codiertes Literal | objectId       // ermöglicht Inversion +
Konflikterkennung
  valueNew    : codiertes Literal | objectId
  state       : vollständige Feature-Map          // nur KEYFRAME (§10)
```

Werte sind codierte Literale (über ConverterService /emf.codec); Referenzen tragen die Ziel-Objekt-ID. valueOld macht jeden Eintrag invertierbar (Undo = old/new tauschen, ADD/REMOVE umkehren) und liefert billige Konflikterkennung. valueOld ist immer der letzte gespeicherte Wert (regelrelativer Abschluss, §8.1 R4): pro (objectId, featureId, address) entspricht er dem valueNew des Vorgängereintrags.

Man beachte, dass der **ChangeSet-Header bereits der Audit-Record auf Objektebene ist**: „wer hat dieses EObject wann und warum geändert“ ist die Folge der Batch-Header seines Streams — ein separater Objekt-Stream existiert nicht und wird nicht gebraucht (er wäre doppelte Buchführung und verletzte P1). Die Objekthistorien-Sicht ist eine Projektion über Header (§15); Audit ohne Werterfassung ist ein klassenweiter Tracking-Modus (§8).

5.2 Delta-Arten

kind	Bedeutung	Inverse
CREATE	Genesis eines Objekts (ID, Klasse; Initialwerte dürfen als SETs im selben Batch folgen)	DELETE

kind	Bedeutung	Inverse
<code>SET / UNSET</code>	einwertiges Feature	<code>SET / UNSET</code> mit getauschten Werten
<code>ADD / REMOVE</code>	mehrwertiges Feature, an <code>index</code>	<code>REMOVE / ADD</code>
<code>MOVE</code>	mehrwertiges Feature, <code>index</code> → <code>toIndex</code>	<code>MOVE</code> zurück
<code>PUT</code>	Map-Feature, adressiert über <code>key</code>	<code>PUT</code> mit getauschten Werten; war der Key abwesend: <code>REMOVE_KEY</code>
<code>REMOVE_KEY</code>	Map-Feature, Key entfernt	<code>PUT</code> , das den alten Wert wiederherstellt
<code>SET_AT</code>	Array-Zelle am Koordinatenvektor <code>coords</code>	<code>SET_AT</code> mit getauschten Werten
<code>RESHAPE</code>	Änderung der Array-Dimensionalität (<code>dimsOld</code> → <code>dimsNew</code>)	<code>RESHAPE</code> zurück
<code>DELETE</code>	Tombstone; Objekt verlässt den Stream	<code>CREATE</code> + Replay (oder Keyframe-Restore)
<code>KEYFRAME</code>	vollständiger getrackter Zustand eines Objekts bei dieser Sequence (§10)	n/a (informativ)
<code>TOUCH</code>	Audit-Modus <code>TOUCHED_FEATURES</code> (§8): Feature wurde geändert, Werte ausgelassen	n/a (informativ)
<code>MIGRATE_OUT</code>	Objekt verlässt diesen Stream (in ein anderes Aggregat verschoben); <code>valueNew</code> = Ziel-streamId (§5.4e)	gespiegeltes <code>MIGRATE_IN / OUT</code> -Paar
<code>MIGRATE_IN</code>	Objekt betritt diesen Stream per Migration; <code>valueOld</code> = Herkunft <code>streamId#seq</code> ; gefolgt von einem <code>KEYFRAME</code> (§5.4e)	gespiegeltes Paar

5.3 Durchgerechnetes Beispiel (einzelnes Objekt)

Modell-Genesis (Metadaten-Kette, §6.4): `EClass 0 (Sensor)` mit `Feature 0 (id)`, `Feature 1 (temp)`, `Feature 2 (hum)` — Modell-Sequence 0, Fingerprint `fpA`.

Instanz-Stream von Sensor `42`:

```
seq 0  fp=fpA  obj=42  CREATE  (class 0)
seq 0  fp=fpA  obj=42  SET    f1   old=n/a  new=6           // selbes ChangeSet wie CREATE
seq 1  fp=fpA  obj=42  SET    f2   old=n/a  new=10
seq 2  fp=fpA  obj=42  SET    f1   old=6     new=12
seq 3  fp=fpA  obj=42  SET    f1   old=12    new=50
```

Jeder Eintrag ist eigenständig interpretierbar: `(fpA, f1)` löst über die Snapshot-Registry zu „Sensor.temp, double, °C“ auf — unabhängig davon, wie das Modell heute aussieht.

5.4 Referenzen und Verschachtelung: die vollständige Taxonomie

Ecore-Referenzen spannen mehrere unabhängige Achsen auf, und das Log muss jede Kombination explizit behandeln: Containment vs. Cross-Referenz, ein- vs. mehrwertig, `ordered`/`unique`-Varianten, unidirektional vs. bidirektional (`eOpposite`), Verschachtelungstiefe (Containment in Containment), Selbstreferenzen (mit Zyklen) und aggregatübergreifende Referenzen.

Grundregeln:

1. **Jedes stream-adressierbare Objekt hat eine stabile ID** gemäß Identitätsvertrag (§4.1: `NATURAL`, `SLOT`, `SYNTHETIC` — oder keine für `EMBEDDED`-Wertkinder, die nur als codierte Werte erscheinen). *Freie* Pfadadressierung („drittes Element von `locations`“) bleibt verboten — sie bricht beim Umsortieren; `SLOT`-Identität ist die disziplinierte Ausnahme, wo der Slot die Domänenidentität *ist*.
2. **Einträge sind flach; Containment ist ein Referenz-Delta.** Ein Containment-Feature hält Kind-Objekt-IDs wie jede Referenz. Die Baumform wird aus Containment-Deltas rekonstruiert.
3. **Stream-Granularität: ein Stream pro Aggregate Root** (pro Top-Level-Objekt, das die Persistenzschicht speichert). Kind-Einträge tragen ihre eigene `objectId`, leben aber im Stream der Wurzel. So bleiben Sequences und Gap-Erkennung pro Aggregat und Batches atomar innerhalb eines Streams. **Aggregatübergreifende Batches** (ein Editor-Save berührt zwei Wurzeln) werden in einen Batch pro Stream aufgeteilt, verknüpft über eine gemeinsame Batch-/Transaktions-ID — Atomarität über Streams hinweg ist dann eine Transaktionsfrage des Backends, nicht des Log-Formats (offene Frage §17).

Pro Achse:

(a) Einwertiges Containment. `SET` mit Kind-ID. Das Ersetzen eines Kindes (`old⇒A new⇒B`) *verdrängt* A — EMF entfernt es aus dem Container. Der Batch muss die Konsequenz explizit machen: entweder `DELETE`-Tombstones für As Teilbaum oder As Wieder-Anhängen anderswo (impliziter Move, siehe (e)). Nie implizit — eigenständige Interpretierbarkeit und Invertierbarkeit dürfen nicht von EMFs Live-Invariantendurchsetzung zur Apply-Zeit abhängen.

(b) Mehrwertige Features (Containment oder Cross). `ADD`/`REMOVE`/`MOVE` mit `index`.

Die Ecore-Flags zählen:

- `ordered=true`: Indizes sind Semantik; Einträge müssen in Reihenfolge angewendet werden (strikte Fault-Policies, §11).
- `ordered=false`: der Index ist ein Speicherartefakt — `ADD`/`REMOVE` **kommutieren**; entspanntere Fault-Policies und Out-of-Order-Anwendung werden für diese Features legal.
- `unique=false`: Listen dürfen Duplikate halten — `REMOVE` adressiert immer über den Index, nie über den Wert.

(c) Bidirektionale Referenzen (`eOpposite`). EMF pflegt die Gegenseite automatisch; beide Seiten naiv aufzuzeichnen würde jede Änderung verdoppeln und Apply brechen (nachdem der erste Eintrag die Gegenseite auto-aktualisiert hat, schlägt der `valueOld`-Guard des zweiten Eintrags fehl). Regel: **genau eine kanonische Seite wird aufgezeichnet**; die Gegenseite ist abgeleitet

— beim Apply durch EMFs eigenes eOpposite-Handling, in Queries/Projektionen durch den Leser.
Kanonische Wahl:

- Containment/Container-Paare: die **Containment-Seite**. `eContainer` wird nie aufgezeichnet.
- Nicht-Containment-Paare (inkl. many-to-many): eine deterministische Regel — Vorschlag: die Seite mit der **niedrigeren stabilen featureId**. Die Wahl wird im Tracking-Aspekt fixiert und ist Teil des Interpretationskontexts (§6): Leser müssen wissen, welche Seite die Wahrheit trägt.

Zwei Konsequenzen, entschieden und empirisch validiert (§22 Entscheidung #3, Dynamic-EMF-Notification-Test vom 18.07.2026; Mechanismus: `EcoreEList.eInverseAdd`, EMF-Quellen):

- **Capture normalisiert.** EMF feuert Notifications auf *beiden* Seiten einer Bidi-Änderung (eine logische Änderung = zwei Events; die Reihenfolge hängt vom Mutationspfad ab). Das Capture zeichnet die kanonische Seite auf, egal welche Seite der Code mutiert hat, und darf nicht von der Notification-Reihenfolge abhängen.
- **Aggregatübergreifende Bidi-Paare: die Gegenseite ist eine Projektion.** Liegt die kanonische Seite im Stream eines anderen Aggregats, rekonstruiert das Replay eines einzelnen Streams das Gegenseiten-Feature *unvollständig* — by design. `get-full` /Resync bedient daher immer die Ist-Zustands-Projektion (vollständig, weil Apply eOpposites pflegt), nie ein Einzelstream-Replay; Queries auf eingehende Referenzen nutzen den Index. Das Tooling warnt, wenn die kanonische Seite eines Bidi-Paars außerhalb des Aggregats des Referenzierenden liegt, damit der Modellierer die kanonische Seite bewusst wählen kann.

(d) Containment in Containment (tiefe Verschachtelung). Die Verflachung behandelt beliebige Tiefe — jede Ebene ist nur ein weiteres Objekt mit eigenem `CREATE` und einem Containment-Delta auf seinem Elternteil. Containment kann nicht zyklisch sein (EMF-Invariante), also existiert immer eine topologische Ordnung: **Erzeugung parent-first, Löschtombstones child-first**, alles innerhalb eines atomaren Batches.

(e) Die Single-Container-Invariante (implizite Moves). Ein Objekt hat genau einen Container. Kind X zu B hinzuzufügen, während X in A enthalten ist, ist ein impliziter Move — das Log macht ihn explizit: `REMOVE` aus `A.f` + `ADD` zu `B.g` in einem Batch (oder ein explizites objektübergreifendes `MOVE`-Delta, offene Frage §17). Dieselbe Regel für die einwertige Verdrängung in (a). Das deckt Moves *innerhalb* eines Aggregats ab; gehören alter und neuer Container zu **verschiedenen Aggregaten**, spannt der Move zwei Streams auf und wird zu einem **Migrationspaar** (entschieden, §22 Entscheidung #4), verknüpft über `transactionId`:

```
Stream S42 (Batch a, Txn T):           Stream S43 (Batch b, Txn T):
  obj=42 SET f3 old=>99 new=null        obj=99 MIGRATE_IN valueOld="S42#7"
  obj=99 MIGRATE_OUT valueNew="S43"    obj=99 KEYFRAME state=[lat=52.51, lon=13.40]
                                       obj=43 SET f3 old=null new=>99
```

`MIGRATE_OUT` wirkt als Tombstone im Quell-Stream (mit Vorwärtsreferenz), `MIGRATE_IN` als Genesis im Ziel-Stream (mit Herkunftsreferenz); der `KEYFRAME` macht den empfangenden Stream replay-self-contained. Historien-Queries folgen den Herkunfts-/Vorwärtsreferenzen über Streams hinweg. Regeln:

- **Ein Paar pro migriertem Objekt — Kinder werden nie impliziert.** Ein migrierender Teilbaum

bekommt explizite Paare für *jedes* enthaltene Objekt (sonst müsste eine Pro-Objekt-Historien-Query die historische Containment-Topologie rekonstruieren, nur um den richtigen Stream zu finden). `MIGRATE_OUT`-Einträge child-first (wie Löschung), `MIGRATE_IN` + `KEYFRAME` parent-first (wie Erzeugung).

- **Containment-Feature-Deltas sind orthogonal** und nutzen den normalen Adressierungsmodus des Features (§5.4b/h/i): einwertig `SET`, geordnete Liste `REMOVE index=2 / ADD index=0`, Map `REMOVE_KEY / PUT`. Migrations-Arten betreffen die Stream-Zugehörigkeit, nie Feature-Werte.
- **Wurzeln dürfen absteigen, Kinder dürfen aufsteigen.** Stream-Eigentum folgt dem Containment-Zustand der Instanz: eine in ein anderes Aggregat degradierte Wurzel migriert (mit ihrem Teilbaum) in den Stream des Wirts und beendet ihren eigenen Stream mit `MIGRATE_OUT`s; ein zur Wurzel beförderter Kindknoten `MIGRATE_IN`t als Genesis eines brandneuen Streams. `ClassTracking.aggregateRoot` deklariert die *Fähigkeit* (dürfen Instanzen einen Stream besitzen), nicht den Pro-Instanz-Zustand.
- **Kontextauflösung ist einheitlich:** beide Batches stempeln dieselbe Composite-Root (§6.4), also lösen Klassen-/Feature-IDs auf beiden Seiten identisch auf.
- **Undo einer Migration** ist das gespiegelte Paar, wieder transaktionsverknüpft über beide Streams.

(f) Selbstreferenzen und Zyklen. Selbst-Containment (`Category.subCategories: Category`)

ist gewöhnliche Verschachtelung — ein selbstverschachtelter Baum ist ein Aggregat, Tiefe unbegrenzt. Nicht-Containment-Selbst- oder Cross-Referenzen können **Zyklen** bilden (`Person.knows: Person`); eine topologische Eintragsordnung existiert dann nicht. Regel: Batches werden in **zwei Phasen** angewendet — Phase 1: alle `CREATE`s; Phase 2: alle Feature-Deltas. Vorwärtsreferenzen innerhalb eines Batches sind legal (das Deferred-Constraints-Muster). Das subsumiert die Ordnungsregel aus (d) und macht ein Objekt, das sich selbst referenziert, trivial.

(g) Aggregatübergreifende Referenzen. Eine Nicht-Containment-Referenz auf eine andere Wurzel trägt die Objekt-ID des Ziels; das Ziel lebt in einem *anderen Stream*. Apply verlangt nicht, dass das Ziel existiert (Proxy-Semantik) — Streams synchronisieren unabhängig, also ist aggregatübergreifende Konsistenz per Konstruktion eventual. Hängende Referenzen sind daher eine konfigurierbare *Policy*, keine Fehlerklasse: bei Ziel-`DELETE`, pro Referenz — `UNSET / REMOVE` des Werts beim Referenzierenden (das FK-Nullify-Analogon), `CASCADE` (auch den Referenzierenden tombstonen; selten) oder `KEEP` (hängend erlaubt, lazy aufgelöst, Queries sehen einen Proxy). Konfiguriert im Tracking-Aspekt; interagiert mit den Fault-Policies (§11).

(h) Maps (EMap). In Ecore ist eine EMap technisch ein mehrwertiges Containment von

`Map$Entry`-EClasses (vgl. `StringToStringMap` im sensinact-mapping). Sie wörtlich zu loggen (Entry-Objekt-`CREATE` + Key/Value-`SET`s + `ADD`) funktioniert, ist aber verrauscht und — schlimmer — *verliert die Map-Semantik*: Maps sind **key-adressiert**, und indexbasierte Konflikterkennung ist für sie schlicht falsch. Regel: der Tracking-Aspekt markiert solche Features als `MAP`, und Capture emittiert erstklassige Map-Deltas —

`PUT(key, valueOld, valueNew) / REMOVE_KEY(key, valueOld)`. Keys sind codierte Literale (oder Objekt-IDs bei Objekt-Keys); Containment-Werte bekommen weiterhin ihr eigenes `CREATE / DELETE` wie jedes Kind (§5.4a gilt pro Key). Entry-EClasses brauchen dann keine

synthetischen IDs und keine Genesis. `PUT`s auf verschiedene Keys kommutieren; gleicher Key = LWW/Konflikt, genau wie `SET`.

(i) Arrays, auch mehrdimensional. Ecore hat kein natives mehrdimensionales Feature — Arrays erscheinen entweder als EAttribute mit Array-/Tensor-Datentyp oder als geschachtelte Listen über Zwischen-EClasses (was sich auf (b)+(d) reduziert). Für echte Array-Features wählt der Tracking-Aspekt eine von zwei Repräsentationen:

- `ATOMIC` (Default): das ganze Array ist ein Wert — schlichtes `SET` mit dem codierten Array als Literal. Richtig für die Sensorwelt, wo Arrays Messrahmen sind (Spektren, Bilder, Matrizen), die pro Zeitstempel als Ganzes ersetzt werden; im `TIMESERIES`-Profil ist jedes Sample der ganze Frame, und Delta-Kompression auf Speicherebene ist eine Backend-Frage, keine Log-Frage.
- `ELEMENT_WISE`: Pro-Zelle-Deltas `SET_AT(coords[n], valueOld, valueNew)` mit einem Koordinaten-Vektor (n-dimensional), plus `RESHAPE(dimsOld, dimsNew)` für Dimensionsänderungen. Richtig für große, langlebige, editierbare Grids. `SET_AT`s auf verschiedene Koordinaten kommutieren; die deklarierte Dimensionalität / der Elementtyp ist Teil des Interpretationskontexts (§6).

Adressierungsmodi — die vereinheitlichende Sicht. (b), (h) und (i) sind ein Konzept: ein Delta adressiert einen *Slot innerhalb eines Features*, und der Adressierungsmodus bestimmt Kommutativität und Konfliktgranularität einheitlich:

Feature-Form	Adresse	Deltas	verschiedene Adressen kommutieren?
einwertig	—	<code>SET</code> / <code>UNSET</code>	n/a (gleicher Slot = LWW/Konflikt)
geordnete Liste	<code>index</code>	<code>ADD</code> / <code>REMOVE</code> / <code>MOVE</code>	nein (Indizes verschieben sich)
ungeordnet mehrwertig	der Wert selbst	<code>ADD</code> / <code>REMOVE</code>	ja
Map	<code>key</code>	<code>PUT</code> / <code>REMOVE_KEY</code>	ja
Array	<code>coords[n]</code>	<code>SET_AT</code> / <code>RESHAPE</code>	ja (<code>RESHAPE</code> konfligiert mit allem)

Konflikterkennung (§5.5) und die Kommutativitätsregeln für Fault-Policies (§11) operieren immer auf `(objectId, featureId, address)` — eine Regel, fünf Formen.

Beispiele. Teilbaum-Erzeugung — Sensor bekommt ein geschachteltes `GeoLocation`-Kind (Feature 3 = `location`, Containment, mit eOpposite `GeoLocation.sensor`):

```
ChangeSet seq 4 (ein Batch, atomar, Zwei-Phasen-Apply):
  obj=99  CREATE  (class GeoLocation, synthetische ID 99)  // Phase 1
  obj=99  SET    lat    old=n/a  new=52.51                // Phase 2
  obj=99  SET    lon    old=n/a  new=13.40
  obj=42  SET    f3     old=null  new=>99                 // kanonische Containment-Seite;
                                                         // GeoLocation.sensor ist abgeleitet, nie
geloggt
```

Impliziter Move *innerhalb eines Aggregats* — Location 99 wandert zwischen zwei Containment-Features desselben Sensors (f3 `location` → f4 `previousLocations`):

```
ChangeSet seq 7 (ein Batch, ein Stream):
  obj=42  SET  f3    old=→99  new=null      // explizite Entfernung aus dem alten Feature
  obj=42  ADD  f4    index=0  →99           // Anhängen an das neue Feature
  // kein DELETE: 99 überlebt, es ist umgezogen. Invertierbar, eigenständig
  interpretierbar.
```

(Ein Move zwischen zwei *Aggregaten* — z. B. Sensor 42 → Sensor 43, beide Wurzeln — ist stattdessen ein Migrationspaar, siehe (e): zwei Streams, zwei Batches,

`MIGRATE_OUT` / `MIGRATE_IN` + `KEYFRAME`.)

5.5 Batching und selektive Anwendung

Ein `ChangeSet` = ein Commit (Editor-Save: mehrere Features ändern sich „gleichzeitig“).

Coalescing (normativ, entschieden — §22 Entscheidung #8). Ein `ChangeSet` enthält **höchstens einen Eintrag pro** `(objectId, featureId, address)`: `valueOld` = der Wert *vor* dem Batch,

`valueNew` = der Wert *danach*. Batch-interne Zwischenzustände (`temp` innerhalb einer Unit of Work `12 → 15 → 14` gesetzt) waren nie beobachtbare Realität — das Log verzeichnet `12 → 14`, spiegelbildlich zur Idee des regelrelativen Abschlusses (§8.1 R4) auf Batch-Granularität.

Netto-Null-Änderungen (`old == new` nach dem Coalescing) werden gar nicht geschrieben.

Ausnahme: geordnete Listen-Features — Index-Operationen kommutieren nicht, ihre Reihenfolge *ist* die Semantik, also werden `ADD` / `REMOVE` / `MOVE`-Sequenzen auf einem geordneten Feature als geordnete Operationsliste im Batch behalten (§5.1 ordnet Einträge ohnehin). `CREATE` + initiale `SET`s in einem Batch (§5.3) bleiben unberührt — Coalescing betrifft nur wiederholte Änderungen an derselben *Adresse*.

Anforderungen aus emf.edit-artigem Tooling:

- **Einen ganzen Batch rückgängig machen:** alle Einträge invertieren, in umgekehrter Reihenfolge anwenden.
 - **Einen einzelnen Eintrag mitten aus der Historie rückgängig machen / anwenden:** möglich, weil Einträge invertierbar sind — verlangt aber eine **Konfliktprüfung**: hat ein späterer Eintrag dieselbe `(objectId, featureId, address)` berührt (§5.4 Adressierungsmodi — geordnete Listen: überlappender Indexbereich; ungeordnete Features: gleicher Wert; Maps: gleicher Key; Arrays: gleiche Koordinaten, `RESHAPE` konfiguriert mit allen Zellen; Bidi: die kanonische Seite prüfen, §5.4c)? Falls ja → mit Diagnostics ablehnen oder an eine Merge-Entscheidung übergeben. Das ist die eine Stelle, an der EMF Compares Konfliktlogik zurückkehrt, klein geschnitten. Nie „apply and hope“.
-

6. Interpretationskontext & Fingerprint

6.1 Das Problem, präzise

Ein Log-Eintrag `(f1, old=12, new=50)` ist nur relativ zu einem Modell-/Metadaten-Zustand bedeutungsvoll. Zwei Änderungen verändern diese Bedeutung:

- **Modell-Refactoring.** In Fortsetzung von §5.3: ein (diszipliniertes) Rename `temp → temperatur` ist ein reines Metadaten-Ereignis — Feature 1 behält seine ID, alle Log-Einträge bleiben gültig, in kein Objekt-Log wird etwas geschrieben. Aber eine *undisziplinierte* Änderung — das Umbinden von Feature-IDs, z. B. das Vertauschen von `1↔temp, 2↔hum` zu `1↔hum, 2↔temp` — verwandelt „seq 2: f1=12 (12 °C)“ und „seq 3: f1=50“ stillschweigend in *12 Grad gefolgt von 50 % Luftfeuchte*. Liest man nur den Objekt-Stream, ist die Korruption unsichtbar.
- **Metadatenänderungen mit semantischer Kraft.** Die Einheit von `temp` von °C auf °F zu ändern ist *dieselbe Problemklasse wie ein Rename*: es ändert die Interpretation historischer Werte ohne jeden Objekt-Log-Eintrag. Maßeinheiten sind keine kosmetischen Metadaten mehr, sobald Historien existieren.

Beides wird einheitlich von einem Prinzip abgedeckt: *Modelle sind für die Instanz nur Metadaten*. Der **Interpretationskontext** ist die Menge aller Metadaten, deren Änderung das Lesen historischer Werte verändert — Ecore-Struktur, Feature-ID-Bindung, Datentypen, Einheiten, Codec-Mappings, orm-Mappings, die die Wertcodierung beeinflussen. (Im Zweifel: rein damit — eine überflüssige Fingerprint-Änderung ist harmlos; eine fehlende nicht.)

6.2 Der Fingerprint-Vertrag

Der Mechanismus ist bewusst abstrahiert (Hash, Merkle-Root, strukturierter Schlüssel — Implementierungsdetail hinter einem Service-Interface). Der Vertrag besteht aus drei Eigenschaften:

1. **Reproduzierbar** — deterministisch aus der kanonischen Form des Kontexts berechenbar. Gleicher Metadaten-Zustand ⇒ gleicher Fingerprint, auf jedem Knoten, ohne Koordination. (Erfordert eine spezifizierte kanonische Serialisierung: Elementreihenfolge, Default-Behandlung.)
2. **Identifizierend** — verschiedene Zustände ⇒ verschiedene Fingerprints, vernachlässigbare Kollisionswahrscheinlichkeit. Größenabschätzung: die Zahl *verschiedener* Metadaten-Zustände über die Lebenszeit eines Systems ist winzig (Dutzende–Tausende); ein 8-Byte-trunkierter Content-Hash ist dafür kollisionssicher und kostet einen long pro Eintrag — nach spaltenweiser Kompression (Millionen identischer Werte) effektiv nichts.
3. **Auflösbar** — Fingerprint → vollständiger Zustand samt Provenienz, über den Snapshot-Store (§6.4). Ein Fingerprint allein identifiziert; er *beschreibt* nie. „Der Fingerprint sagt, was sich geändert hat“ stimmt nur als Fingerprint + Snapshot-Registry + Diff.

Das ist das Schema-Fingerprint-Muster, bekannt aus der Kafka Schema Registry (jede Nachricht trägt eine Schema-ID; Consumer lösen gegen die Registry auf).

6.3 Jeder Eintrag trägt den Fingerprint

„Abwesenheit heißt unverändert“ (Delta-Codierung des Fingerprints) wurde erwogen und **als logische Regel verworfen**: sie macht Einträge kontextabhängig (das Interpretieren von seq 3 erfordert Rückwärtsscannen bis zum letzten Eintrag *mit* Fingerprint), bricht wahlfreien Zugriff auf Log-Bereiche und — am schlimmsten — bricht die Retention: eine `DeletionRule` („letzte 100 behalten“) löscht irgendwann genau den Eintrag, der den letzten Fingerprint trug, und lässt den überlebenden Kopf uninterpretierbar zurück.

Auflösung: **logisch hat jeder Eintrag einen effektiven Fingerprint; physisch darf das Speicherformat ihn lauflängen-/delta-codieren**. Dieselbe Datenminimierung, aber als Eigenschaft der Codierung, nicht der Semantik. (In spaltenorientierter Speicherung komprimiert die Fingerprint-Spalte ohnehin gegen null.)

6.4 Snapshot-Kette = der Modell-Stream

Jeder Metadaten-Snapshot wird content-addressed gespeichert (unter seinem Fingerprint) und **bettet den Fingerprint seines Vorgängers ein** (das Git-Commit-Prinzip). Folgen:

- Die Provenienzkette steckt im Snapshot-Store selbst — **der „Modell-Stream“ ist keine separate Struktur**, er *ist* die verkettete Snapshot-Folge.
- „Version einer EClass“ ist eine **Position in der Kette**, kein Attribut. Keine doppelte Buchführung (ein Versionsattribut kann von den tatsächlichen Änderungen divergieren; die Kette kann das nicht — sie *ist* die Änderungen).
- Jeder historische Modellzustand ist adressierbar; „Modell zum Zeitpunkt X“ ist wohldefiniert.
- Menschenlesbare Versionen („1.2“) sind **Tags/Labels auf Kettenpositionen** (das Git-Tag-Prinzip). Maschinen vergleichen Fingerprints/Positionen; Release-Management hängt Namen an.
- **Bootstrap**: der initiale Modellimport schreibt einen Genesis-Snapshot („EClass Sensor erzeugt mit Features ...“). Ohne ihn heißt „ältester Eintrag“ nur „seit wir tracken“, nicht „seit es existiert“. Dasselbe Genesis-Prinzip gilt für Objekt-Streams (`CREATE`).
- Snapshots, die von irgendeinem überlebenden Log-Eintrag referenziert werden, werden **nie garbage-collected**.

Fingerprint und Kette sind kein Entweder-oder — sie sind **Identität vs. Ordnung**: der Fingerprint sagt (koordinationsfrei), *welcher* Zustand; die Kette sagt, *in welcher Reihenfolge* und *über welchen Pfad*. Git hält aus gutem Grund beides — Content-Hashes *und* Parent-Pointer.

Modell-Governance ist zentralisiert (normativ, §22 Entscheidung #6). Die Kette ist eine lineare Liste — es gibt bewusst kein Fork/Merge (keinen DAG). Ein Schreiber pro Paket-Kette; ein Publish, dessen Vorgänger nicht der aktuelle Head ist, wird **abgelehnt** („fast-forward only“, wie `git push --ff-only`), und der Autor rebased. Modellevolution ist ein bewusster Governance-Akt, nie ein nebenläufiger Schreibpfad.

Intern wird der Fingerprint als **Merkle-Baum** strukturiert — Feature-Hashes → EClass-Hashes → Paket-Root, und darüber die **Composite-Root**: ein Hash über die geordnete Liste der `(packageUri, packageFingerprint)`-Paare aller Pakete im Deployment-Scope.

ChangeSets stempeln die Composite-Root (entschieden 18.07.2026, §22 Entscheidung #2): ein

Batch darf Klassen aus mehreren Paketen berühren (paketübergreifendes Containment innerhalb eines Aggregats), und ein einziger Composite-Stempel hält jeden Eintrag auflösbar — `(compositeFp, classId)` → Paketzustand → Feature — vorausgesetzt, classIds sind *registry-weit* eindeutig, nicht bloß pro Paket (Tracking-Aspekt). Der `CompositeSnapshot` ist winzig (die Paarliste), content-addressed und vorgängerverkettet wie alles andere; **die Pro-Paket-Ketten bleiben die Einheiten von Evolution und Governance** — das Composite referenziert lediglich ihre Zustände. Single-Package-Deployments degenerieren sauber: das Composite hat einen Eintrag und ändert sich genau dann, wenn der Paket-Fingerprint es tut.

„Was genau hat sich zwischen zwei Composite-Zuständen geändert" ist ein billiger Baumvergleich mit Drill-down (Composite → Paket → EClass → Feature), sodass ein Leser präzise erfährt, ob *seine* Einträge betroffen sind. Das macht auch die Unterscheidung log-relevanter/log-neutraler Metadaten (Docs vs. Units) weitgehend selbstbeantwortend: eine Dokumentationsänderung ändert ein Blatt, das kein Leser von Wertsemantik konsultiert.

6.5 Drei Verteidigungsschichten (Zusammenfassung)

Schicht	Rolle	Deckt ab
Feature-IDs	Disziplin — macht den Normalfall (Rename) zum Nicht-Ereignis	Renames, Namensverschiebungen
Kontext-Fingerprint	Sicherheitsnetz — macht <i>jede</i> Änderung erkennbar	Einheitenänderungen, Disziplinverstöße, alles Unvorhergesehene
Snapshot-Kette	Gedächtnis — macht Änderungen <i>interpretierbar</i> (Upcasting-Pfad von fpA nach fpB)	Typänderungen, Splits/Merges, Einheitenkonvertierungen

Alte Einträge unter neuem Modell lesen = **Upcasting on read** (das Event-Sourcing-Muster): Fingerprint des Eintrags auflösen → Diff zum aktuellen Zustand → Konverter (z. B. °C→°F) on the fly anwenden. Optional darf das Housekeeping alte Einträge beim Retention-Cleanup auf den aktuellen Kontext **kompaktieren** (Transform-on-write, amortisiert) — eine Optimierung, nie eine Anforderung.

7. Capture: der Hybrid

Zwei Capture-Quellen, ein Ausgabeformat (ChangeSets), eine nachgelagerte Pipeline.

7.1 SnapshotCapture (state-basierter Ingest) — der Normalfall

Szenario: TTN-JSON kommt an, wird in ein Sensor-EObject deserialisiert — ein *vollständiger neuer Zustand* (neue temp, Location, Geräte-Zeitstempel). Es gibt keine Change-Events zum Aufzeichnen; die ChangeRecorder-Architektur trägt hier nicht. **Diese Fälle sind sehr häufig.**

Der nötige Vergleich ist *kein* EMF-Compare-artiger, denn **die Identität ist bekannt** (die Sensor-ID steht im Payload). EMF Compares Kernkosten — das Matching, welches Objekt welchem entspricht — entfallen vollständig. Was bleibt, ist ein **keyed Snapshot-Diff**: letzten bekannten Zustand laden (Cache/DB), *nur die getrackten Features* vergleichen, flach, typisiert

— $O(\# \text{gettrackte Features})$, kein Baum-Matching. Die Deltas als ein ChangeSet emittieren.

Ein Vorbehalt, aufgelöst durch den Identitätsvertrag (§4.1): für **Containment-Kinder** braucht das keyed Diff die Identität des Kindes. Einwertig/`SLOT` ist trivial (der Slot korreliert); mehrwertig erfordert `NATURAL`-matchKeys — ohne sie degradiert Capture zu Replace-all (REMOVE all + ADD all, Log-Churn), und das Tooling warnt; `EMBEDDED` umgeht das Matching komplett per Wertvergleich.

Eigenschaften:

- **Selbstheilend**: state-basierter Transfer heißt, eine verlorene Nachricht kostet ein Sample, nicht die Konsistenz — der nächste Full State repariert alles. „Kein Change-Event darf verloren gehen“ gilt nur für eventbasierte Systeme; hier ist der Zustand die Wahrheit, nicht das Event.
- **Degeneriert zu Append** für hochfrequente numerische Features mit TIMESERIES-Profil: ChangeRules-Filter anwenden, Sample anhängen, Ist-Wert setzen — gar kein Diff.

7.2 RecordingCapture (Editieren / programmatische Schreibzugriffe)

Ein Recorder (ChangeRecorder-artiger Adapter) puffert in eine Unit of Work und emittiert das ChangeSet **erst beim Commit** — CommandStack-Semantik. Das beantwortet die Sorge um Rauschen/unerwünschte Events: es ist ein Commit-Grenzen-Problem, kein Recorder-Problem. Kein systemweiter Always-on-Recorder; einer pro Editing-Session/Transaktion. Der Recorder **normalisiert Bidi-Notifications auf die kanonische Seite** (§5.4c): EMF feuert Events an beiden Enden eines eOpposite-Paars, in mutationspfadabhängiger Reihenfolge — eine logische Änderung muss genau einen Eintrag ergeben.

7.3 Gemeinsame Pipeline

```
SnapshotCapture ┌
                  │
                  └─→ ChangeRule-Filter (Deadband/Throttle, §3.6) ─→ Senken:
RecordingCapture ┌
                  │
                  └─→
(CHANGELOG / TIMESERIES)                                • Ist-Zustand (wie heute)
                                                           • Change-Stream
```

Verteilungsregel (P3): Change-Events reisen nie als Quelle der Wahrheit zwischen Systemen. Capture liegt an der Persistenzgrenze; Verlustfreiheit ist nur innerhalb einer Transaktion gefordert. Knoten-übergreifende Synchronisation = state-basiert oder Replay aus dem persistierten Log (§12).

8. Tracking-Konfiguration (der Aspekt)

Pro EClass/Feature, in der Metadaten-/eorm-Aspektebene (dem Muster folgend, das die `batch`-Fetch-Property bereits im eorm-Modell etabliert hat):

```
TrackingConfig (pro Klasse)
  audit          : OFF | HEADER | TOUCHED_FEATURES
```

```

// Ganzobjekt-Audit ohne Werterfassung: HEADER zeichnet Wer/Wann/Warum
auf
// (nur Batch-Header); TOUCHED_FEATURES zeichnet zusätzlich auf, WELCHE
// Feature-IDs berührt wurden — aber keine Werte
(Datenminimierung/Privacy).
// Komponiert mit Feature-Tracking: Features dürfen NONE sein, während
Audit
// an ist; bei aktivem Feature-Tracking fügt HEADER nichts hinzu
// (Header existieren immer).

TrackingConfig (pro Feature)
  mode          : NONE | CHANGELOG | TIMESERIES
  changeRules   : ChangeRule[*]      // aus sensinact-mapping gehoben, §3.6
  deletionRule  : DeletionRule       // Retention: Alter und/oder maxCount
  keyframe      : alle N Einträge / alle T Zeit / NONE (§10)
  faultPolicy   : Policy-Overrides pro Stream (§11)
  featureId     : stabile ID-Zuweisung (§4.2)
  shape         : SINGLE | ORDERED_LIST | UNORDERED | MAP | ARRAY (§5.4 Adressierungsmodi)
  arrayMode     : ATOMIC | ELEMENT_WISE // nur ARRAY (§5.4i)
  danglingRefs  : KEEP | UNSET | CASCADE // aggregatübergreifende Referenzen (§5.4g)

```

Konkrete Beispiele aus der Diskussion: Feature A behält die letzten 100 Werte

(`DeletionRule.maxCount=100`); Feature B behält 3 Tage, Feature C 5 Tage

(`retention=3/5 DAYS`); temp nur bei $|\Delta| > 0.5$ tracken (`AbsoluteChangeRule`) oder $\pm 10\%$

(`PercentageChangeRule`).

Privacy-Metadaten sind hier nicht passiv — ihre Rolle ist aber bewusst begrenzt (entschieden, §22 Entscheidung #7): **das Erheben personenbezogener Daten ist legitim; was existieren muss, ist ein Zweck und dessen Dokumentation.** Der Privacy-Aspekt *dokumentiert* daher primär: welche Features personenbezogen sind, den Zweck / die Rechtsgrundlage, wer die betroffene Person ist und die geltenden Direktiven. *Wie* Daten geschützt werden — Verschlüsselung, Anonymisierung, Pseudonymisierung, Aufbewahrungsgrenzen — wird **extern diktiert** (Compliance/DSB/Kunde), nie vom Framework entschieden. Das Framework steuert Mechanik bei, keine Policy:

- Extern diktierte Retention **beschränkt** die Tracking-Config („dieses Feature darf höchstens 30 Tage Historie halten“) — ein Grund mehr, warum beide Aspekte in derselben Registry-Ebene leben.
- **Durchsetzung läuft als separate Jobs:** das DeletionRule-Housekeeping und Backend-TTL-Mechanismen (Mongo-TTL-Indizes, TimescaleDB-Retention-Policies). Gezielte Löschung, Schwärzung und Crypto-Shredding sind *Fähigkeiten, die diese Jobs auf Anweisung ausführen* — nie Inline-Magie im Schreibpfad (Capture darf Werte weiterhin verschlüsseln, wo so angewiesen).
- Ein **Advisor** darf Modelle und Aspekte analysieren und Konfigurationen *vorschlagen* („Feature `location` sieht personenbezogen aus, keine Retention konfiguriert“) — Vorschläge erfordern immer menschliche Freigabe. Privacy-Konfiguration ist nie vollautomatisiert.

8.1 Regelsemantik (normativ)

Diese Bedingungen sind bindend für Capture-, Apply- und Housekeeping-Implementierungen (entschieden 18.07.2026, inkl. Teilentscheidung 1a — siehe §22 Entscheidung #1):

- **R1 — Kein Aspekt, kein Stream.** Eine EClass ohne Tracking-Config bekommt nur Ist-Zustands-Persistenz: kein Stream, kein Audit. Audit ist Opt-in (`AuditMode`).
- **R2 — Kaskadierende Aktivierung.** `mode = CHANGELOG | TIMESERIES` auf irgendeinem Feature eines Pakets aktiviert die Snapshot-Kette dieses Pakets; die erste Aktivierung schreibt automatisch den Genesis-Snapshot (§6.4).
- **R3 — Regeln gaten nur den Stream.** ChangeRules entscheiden, was in den *Stream* gelangt; der Ist-Zustand folgt immer den Live-Werten (die sensiNact-Semantik: Regeln steuern die Weiterleitung in die Historie, nie den Live-Wert).
- **R4 — Regelrelativer Abschluss.** Pro `(streamId, objectId, featureId, address)` entspricht das `valueOld` jedes gespeicherten Eintrags dem `valueNew` seines Vorgängers; der erste gespeicherte Wert hat kein `valueOld`. Verworfen Übergänge sind *unbekannt*, nicht *verloren* — Sampling-Semantik: die Realität zwischen gespeicherten Samples war nie Teil der persistierten Aufzeichnung.
- **R5 — Toleranzbegrenzter Fold.** `fold(stream) ≈ Ist-Zustand` innerhalb der von den aktiven ChangeRules deklarierten Toleranz; ohne Regeln ist die Gleichheit exakt (pro getracktem Feature).
- **R6 — Regeln sind Kontext.** ChangeRules leben im Tracking-Aspekt und sind log-relevant: jede Regeländerung erzeugt einen neuen Snapshot → neuen Fingerprint (§6). Einträge werden unter den Regeln *ihres* Fingerprints interpretiert.
- **R7 — Baseline nach Änderung.** Der erste Wert eines Features wird immer gespeichert; ebenso der erste Wert nach jeder Regeländerung (neuer Fingerprint) — bedingungslos, um die Baseline zu etablieren, gegen die die nächste Regelauswertung vergleicht.
- **R8 — Undo stellt gespeicherte Realität wieder her.** Die Inversion zielt auf den letzten *gespeicherten* Wert — korrekt per Deklaration, denn Regeln sind deklarierte Sampling-Absicht.

9. Storage-Profile

Zeitreihe und Changelog sind **technisch dasselbe** — ein geordneter, versionierter Strom von Änderungen; bei der Serie ist die „Version“ zeitkorreliert. Äquivalent aus der anderen Richtung: *Diff-Persistenz ist eine gebatchte Zeitreihe ohne (oder mit) Zeitelement* — die Version ist ein aufsteigender long: Epoch-Millis, ein Zähler oder bloß die Eintragsreihenfolge. Ein konzeptionelles Modell, zwei Storage-Profile:

	CHANGELOG	TIMESERIES
Granularität	ChangeSets (Multi-Feature-Batches)	Einzel-Feature-Samples
typische Frequenz	niedrig (User-Edits, Importe)	hoch (Sensorwerte)

	CHANGELOG	TIMESERIES
Layout	Dokument/Zeile pro Batch	schmal spaltenorientiert (<code>objectId, featureId, seq, ts, fp, value</code>); Mongo Time-Series-Collections; JPA: schmale Tabelle (pro Serie oder generisch); später TSDB-Backends
Natur der Einträge	Deltas (old/new)	Absolutwerte — jedes Sample ist effektiv sein eigener Keyframe
Verwendung	Audit, Undo, Versionierung, Replikation	Metriken, Dashboards, Analytics

Das Ist-Zustands-EObject ist in beiden Fällen die materialisierte Projektion des Streams (P1); die heutige Persistenz bleibt wie sie ist und *ist* diese Projektion. Zwei Repräsentationen, eine Quelle: Sensor mit aktuellem `temp`-Wert + die Serie für `temp`. Beide Profile sind **verlustfrei relativ zu den geltenden ChangeRules** (§8.1): ein regelfreies CHANGELOG verzeichnet jeden Übergang; mit Regeln ist der Stream die deklarierte gesampelte Realität.

10. Keyframing

Die Video-Codec-Idee (I-Frames): periodisch einen `KEYFRAME`-Eintrag — den vollständigen getrackten Zustand des Objekts bei dieser Sequence — in den Stream schreiben.

Zwecke:

1. **Begrenztes Replay.** „Objekt zum Stand seq N" / `asOf`-Queries starten am letzten Keyframe $\leq N$ und replaysen vorwärts — nicht ab der Genesis.
2. **Retention-sichere Kürzung.** Die `DeletionRule` schneidet *an Manifesten* (unten): alles Ältere darf weg; der Stream-Kopf bleibt self-contained. Würde ein Schnitt das letzte Manifest entfernen, synthetisiert das Housekeeping zuerst eines.
3. **Kompaktierung.** Das Housekeeping darf lange Delta-Läufe, die älter als ein Manifest sind, durch das Manifest selbst ersetzen (Changelog $\rightarrow$ Snapshot + jüngste Deltas).
4. **Resync-Anker** für den Transport (§12): ein Empfänger, der sich von einer Lücke erholt, bekommt „letztes Manifest + Deltas seither" statt der vollen Historie (§11.2).

Stream-Manifeste (entschieden, §22 Entscheidung #5). Pro-Objekt-`KEYFRAME`s können *Abwesenheit* nicht ausdrücken — und „das Aggregat zum Stand seq N" braucht alle Objekte. Ein ChangeSet mit Flag `manifest = true` trägt `KEYFRAME`-Einträge für **jedes lebende Objekt** des Aggregats bei dieser Sequence:

- **Abwesenheit im Manifest = gelöscht.** Tombstones (`DELETE`; `MIGRATE_OUT`, sobald seine Vorwärtskette bestätigt ist), die älter als das Manifest sind, werden purgebar — das klassische Tombstone-Resurrection-Problem löst sich auf, denn „Objekt 77 existiert nicht mehr" ist in der Vollständigkeit des Manifests codiert.
- Das Flag ist **explizit**, nie eine Konvention („Batch, der nur Keyframes enthält"):

Vollständigkeit *ist* die Semantik, und ein partieller Keyframe-Batch darf nie mit einem Manifest verwechselbar sein.

- Die `KeyframeConfig`-Kadenz steuert Manifeste; Pro-Objekt-Keyframes *zwischen* Manifesten bleiben eine Replay-Optimierung.

TIMESERIES-Profil brauchen selten explizite Keyframes — jedes Sample ist ein Absolutwert, also sein eigener Keyframe; deshalb ist das Serienprofil auch von Natur aus fehlertolerant (§11.3).

11. Fehlertoleranz: Ordnung, Lücken, Duplikate

Deltas sind nur anwendbar, wenn der Empfänger den Vorgängerzustand des Streams hält. Die `sequence` (lückenlos pro Stream) macht Anomalien *erkennbar*; **Fault-Policies** definieren die Reaktion. Policies sind pro Stream/Profil konfigurierbar (Defaults unten).

11.1 Anomalien und Policies

Lücke (fehlende Sequence) — seq 7 kommt an, der Head ist 5:

- `BUFFER_AND_WAIT(timeout, maxBuffer)` — 7 festhalten, auf 6 warten (Fall Netzwerk-Umordnung); bei Timeout eskalieren.
- `RESYNC` — Zustand anfordern (§11.2). Default für CHANGELOG.
- `SKIP_AND_MARK` — trotzdem anwenden, einen **Lückenmarker** im Stream verzeichnen. Nur vertretbar für absolutwertige Streams (TIMESERIES) oder wenn ein Keyframe ankommt; die Lücke ist für Queries/Projektionen sichtbar („Daten fehlen zwischen seq 5–7“).
- `QUARANTINE` — Anwenden stoppen, den Stream parken, eine Diagnose erheben. Für Daten, bei denen „nie raten“ gilt.

Out-of-Order (veralteter Eintrag) — seq 6 kommt an, nachdem 7 angewendet wurde:

- innerhalb eines konfigurierten **Reorder-Fensters**: einfügen, ab 6 neu anwenden (oder direkt anwenden, wenn es kommutiert — siehe unten).
- älter als das Fenster: CHANGELOG → Konfliktpfad (kann spätere `valueOld`s invalidieren) oder Drop-with-Audit; TIMESERIES → **akzeptieren** — Samples werden über Event Time adressiert, das Einfügen ist reihenfolgeunabhängig; späte LoRaWAN-Uplinks sind Alltag.

Duplikat — Apply ist **idempotent** über `(streamId, sequence)`: bereits angewendete Sequences werden stillschweigend übersprungen. Das macht At-least-once-Transporte sicher.

Kommutativitätsnotiz: `SET`s von Absolutwerten mit verschiedenen Event Times kommutieren (Last-write-wins nach Event Time für die Ist-Zustands-Projektion; beide landen in der Serie).

Geordnete Listenoperationen (`ADD/REMOVE/MOVE` mit bedeutungstragenden Indizes)

kommutieren **nicht** — Streams, die sie enthalten, müssen strikte Policies verwenden

(`BUFFER_AND_WAIT` → `RESYNC`). Deltas auf **verschiedenen Adressen** kommutieren (§5.4

Adressierungsmodi): `ADD/REMOVE` auf ungeordneten Features, `PUT/REMOVE_KEY` auf

verschiedenen Map-Keys, `SET_AT` auf verschiedenen Array-Koordinaten — diese dürfen entspannte

Policies behalten (`RESHAPE` ist die Ausnahme: es konfiguriert mit jeder Zelle und erzwingt

strikte Behandlung).

11.2 Get-full (State-Resync)

Die universelle Reparatur, und derselbe Mechanismus wie die Erstkontakt-Synchronisation: der Empfänger fordert den **aktuellen vollständigen Zustand** des Aggregats an (oder: letztes Manifest + Deltas seither, wenn er lokale Historienkontinuität bewahren will) zusammen mit der aktuellen Sequence-Wasserlinie, und setzt die Delta-Anwendung von dort fort. Das ist P3s Selbstheilung, auf Replikation angewendet — dasselbe Muster, von dem der TTN-Ingest lebt. Deltas für den Normalfall, State-Sync für Bootstrap und Reparatur: ein Protokoll, kein Sonderfall-Zoo.

11.3 Default-Policy-Matrix

	CHANGELOG	TIMESERIES
Lücke	<code>BUFFER_AND_WAIT</code> → <code>RESYNC</code>	<code>SKIP_AND_MARK</code>
Out-of-Order (im Fenster)	umordnen & neu anwenden	nach Event Time einfügen
Out-of-Order (veraltet)	Konfliktpfad / Drop-with-Audit	nach Event Time einfügen
Duplikat	idempotentes Überspringen	idempotentes Überspringen
Reparaturanker	Manifest + Deltas	n/a (Samples self-contained)

12. Transport & Replikation

12.1 Der stabile Umschlag

Das einzige Schema, das Sender und Empfänger teilen müssen, ist das Diff-Ecore selbst — winzig, generisch, faktisch einfrierbar. Auf der Leitung (Protobuf oder irgendein Codec) ist das Nachrichtenschema das *Patch-Metamodell*, nie das geänderte Domänenmodell. Domänenwerte in Deltas sind codierte Literale, deren Typisierung sich aus `(contextFingerprint, featureId)` zur *Apply*-Zeit auflöst, nicht zur Empfangszeit.

Folgen:

- **Domänenmodell-Evolution berührt die Leitung nie.** Ein Empfänger von 2026 kann Diffs eines Senders von 2030 akzeptieren, ohne dessen Modell zu kennen. Modelländerungen transportieren sich ohne jede Protokolländerung.
- **Store-and-forward gratis.** Ein Empfänger kann Diffs mit unbekanntem Fingerprint annehmen, persistieren und weiterleiten; nur `apply` braucht den Snapshot.
- **Wire-Format = Speicherformat = API-Payload** (das Git-Pack-Prinzip: Objekte auf der Leitung sind die Objekte im Store). Replikation ist „Log kopieren“; Export ist „Log serialisieren“; Audit ist „Log lesen“. Keine Übersetzungsschichten.

12.2 In-Band-Metadaten-Snapshots

Damit Apply nie an einer entfernten Registry blockiert: Metadaten-Snapshots reisen **in-band**, im selben Umschlag — eine Modelländerung ist selbst nur ein Changeset in der Snapshot-Kette, und klein. Die Vorgängerverweise lassen den Empfänger die Kettenvollständigkeit verifizieren; ein fehlendes Glied wird bei Bedarf *gezogen* (nie blind gewartet).

12.3 Die eine Stelle, die manuelle Disziplin braucht

Das Diff-Ecore kann sich nicht selbst fingerprint-bootstrappen (Henne-Ei). Es bekommt die Protobuf-Doktrin von Hand angewendet: stabile Feld-IDs, nur additive Evolution, nie die Semantik eines Feldes umbinden. Daher: bewusst klein und langweilig halten — das Koordinatentupel plus Delta, und jeder Versuchung widerstehen, Convenience-Felder hineinzudesignen. Convenience gehört in Projektionen.

12.4 Replikationsprotokoll (Skizze)

Normalbetrieb	: Subscription auf Pro-Aggregat-Diff-Streams (sequence-lückenlos)
Erstkontakt	: get-full (State oder Keyframe+Deltas) + Wasserlinie, dann Subscription
Lücke erkannt	: Fault-Policy → meist get-full-Resync (§11.2)
Metadaten	: Snapshot-Kette in-band repliziert; Lücken per Vorgängerverweis gezogen
Idempotenz	: (streamId, sequence) — At-least-once-Zustellung ist sicher
Sequencing	: Single-Master — der kanonische Log-Store vergibt Sequences (§4.3); Producer dedupen über ChangeSet.id, Replikate sind read/apply-only
Governance	: ein Schreiber pro Paket-Kette, Publishes nur fast-forward (§6.4)

13. Undo/Redo, selektives Anwenden, Konflikte

- **In-Memory:** emf.edit CommandStack, unverändert — er ist das interaktive Undo innerhalb einer Session.
- **Persistent:** ChangeSets sind das dauerhafte Gegenstück (Batch = Commit). Undo des letzten Batches = Einträge invertieren, umgekehrt anwenden. Redo = erneut anwenden.
- **Selektives Undo/Apply mitten aus der Historie** (Anforderung 6): erlaubt, abgesichert durch die Konfliktprüfung aus §5.5 — ein späterer Eintrag, der dieselbe `(objectId, featureId)` berührt, blockiert stille Anwendung und produziert Diagnostics / eine Merge-Entscheidung. EMF Compares Konfliktlogik ist die Referenzimplementierung für diese Prüfung (ihre einzige tragende Laufzeitrolle).
- **valueOld als Guard:** beim Apply signalisiert ein Eintrag, dessen `valueOld` nicht zum aktuellen Wert passt, Divergenz (Optimistic-Concurrency-Check auf Delta-Granularität) — Policy: fail, force (LWW) oder Three-way-Merge.

14. Query-Integration

- **QueryProcessor-SPI** pro Backend übersetzt das Query-Modell; Capability-Deklaration + Diagnostics bei nicht unterstützten Konstrukten (§3.1).
- **Zeitdimension:** `asOf(sequence | timestamp)` — Auflösung über letzten Keyframe $\leq$ Ziel +

Replay (§10); **Serien-Queries** — Range/Aggregation über TIMESERIES-Streams (die `DateComparator`s existieren; was fehlt, ist der Begriff „das Subjekt ist eine Serie, nicht der Ist-Zustand“).

- **CRUD**: Delete = Selektor; **Update = Selektor + Patch** — ein `UpdateCommand` referenziert eine Query und ein ChangeSet-Template; kein separates Update-Vokabular.
- Gespeicherte Queries (`saveQuery`) persistieren durch dieselbe Maschinerie (P2).

15. Metadaten & Projektionen

- **Registry (API)** — der Metadaten-Service (§3.2) als Single Point of Contact.
- **Index-Holder (Projektion)** — pro EClass/Objekt: Verweise auf Metadaten-Dateien, auf Zeitreihen, auf Snapshot-Ketten-Positionen. Asynchron gepflegt, rekonstruierbar (P1).
- **Abgeleitete Versionen** — „Version einer EClass“ = letzte Kettenposition, die sie berührt hat (eine gefilterte Projektion über die Snapshot-Kette; wohldefiniert, weil die Kette pro Paket total geordnet ist). `changeDate` = jüngster, `creationDate` = ältester (Genesis!) relevanter Ketteneintrag. Gespeicherte Kopien davon sind vom Housekeeping *konsistenzprüfbar* (neu berechnen vs. gespeichert) — es sind Ableitungsregeln, keine Constraints; die Kette bleibt die Wahrheit.
- **Objekthistorie / Audit-Sicht** — „wer hat dieses Objekt wann und warum geändert“ als Projektion über die ChangeSet-Header seines Streams (Werte ausgelassen); die Querschnittsvariante („wer hat heute irgendetwas geändert“) ist dieselbe Projektion über alle Streams. Im Index materialisierbar (Lucene); nie ein zweiter Stream.
- **Künftige Aggregate** (min/max/avg über Serien) — derselbe Mechanismus, anderer Stream: Continuous-Aggregate-Projektionen über Objekt-/Serien-Streams. Bewusst zurückgestellt; das Projektionskonzept hält die Tür offen, zu null Zusatz-Mechanikkosten.

16. Metamodell-Evolution — durchgerechnetes Beispiel

In Fortsetzung von §5.3 / §6.1, mit allem an Ort und Stelle:

1. **Rename** `temp` → `temperatur` (diszipliniert): Metadaten-Snapshot `fpB` geschrieben, Vorgänger `fpA`. Merkle-Diff: Namensblatt von Feature 1. Objekt-Logs: **unberührt** (Feature-IDs). Leser lösen (`fpA`, `f1`) und (`fpB`, `f1`) zum selben semantischen Feature auf. Kosten: ein Snapshot. (Kontrast: das naive Inject-in-Objekt-Logs-Design hätte das bei 1 Mio. Instanzen 1 Mio. Mal geschrieben.)
2. **Einheitenwechsel** °C → °F auf Feature 1: Snapshot `fpC` (Einheitenblatt geändert). Neue Einträge tragen `fpC`. Lesen eines gemischten Bereichs: mit `fpA/fpB` gestempelte Einträge werden on read upcasted (×9/5+32) — oder lazy vom Housekeeping kompaktiert. Queries sehen eine kohärente Serie.
3. **Disziplinverstoß** (Feature-ID-Tausch `1↔hum`, `2↔temp`): Snapshot `fpD`; die Fingerprint-Änderung macht das Umbinden *erkennbar* (Merkle: Semantikblätter von `f1/f2` geändert, nicht nur Namen) — das Tooling kann ablehnen oder ein explizites Migrations-Mapping verlangen. Ohne den Fingerprint korrumpiert das die Historie stillschweigend (12 °C, dann 50 % Luftfeuchte, §6.1).

4. **Strukturänderung** (Feature-Split/-Merge, Typänderung): Snapshot + ein **Upcaster**, registriert für den Übergang $fpX \rightarrow fpY$; Replay/Read wendet ihn an. Das ist die Standard-Upcasting-Kette des Event Sourcing.

17. Offene Entscheidungen

1. **ID-Strategie-Vertrag** — *entschieden* (§22 Entscheidung #9): Identität pro Containment-Feature über `IdentityStrategy` (NATURAL / SLOT / SYNTHETIC / EMBEDDED, AUTO als Default), siehe §4.1. Noch offen: wo die Durchsetzung lebt (orm-Prozessor vs. Laufzeitvalidierung).
2. **Kanonisierungsspezifikation** für den Fingerprint (Elementreihenfolge, Defaults, Codierung) und Algorithmus-/Längenwahl (8-Byte-trunkierter Content-Hash vorgeschlagen; registry-vergebene kompakte lokale IDs nur als *gemessene* Optimierung).
3. **Umfang des Interpretationskontexts** — Ecore-Struktur, Feature-ID-Bindung, Typen, Einheiten, Array-Dimensionalität/-Elementtypen, Map-Key/Value-Typen, Kanonische-Seite-Entscheidungen sind drin; Codec-/eorm-Mappings drin, wenn sie die Wertcodierung beeinflussen; Faustregel: im Zweifel drin.
4. **Kontextgranularität** — *entschieden* (§22 Entscheidung #2): ChangeSets stempeln die **Composite-Root** (Merkle-Root auf Deployment-Ebene über die Pro-Paket-Fingerprints); Pro-Paket-Ketten bleiben die Evolutions-/Governance-Einheiten; classIds sind registry-weit eindeutig.
5. **Atomarität aggregatübergreifender Batches** — gemeinsame Transaktions-ID + Backend-Transaktion vs. Two-Phase auf Log-Ebene; Fähigkeiten pro Backend (JPA ja, Mongo Multi-Dokument-TX eingeschränkt).
6. **Transaktionalität Ist-Zustand + Log** — gleiche TX in JPA (einfach); Mongo: Change-Stream-Eintrag und Ist-Zustands-Dokument in einer Multi-Dokument-TX oder Outbox-Muster.
7. **MOVE über Container hinweg** — explizites objektübergreifendes Delta vs. REMOVE+ADD-Paar.
8. **Wo die neuen Metamodelle leben** — `fennec.common.models` (neben dem Query-Modell) vorgeschlagen: Patch-/Stream-Modell, Tracking-Aspekt (inkl. der aus sensinact-mapping gehobenen Regeln), Snapshot-Ketten-Modell.
9. **Privacy-Durchsetzung** — *entschieden* (§22 Entscheidung #7): der Privacy-Aspekt dokumentiert Zweck, Betroffene und Direktiven; Durchsetzung = separate Housekeeping-Jobs + Backend-TTL-Mechanismen, die extern diktierte Direktiven ausführen; Advisor schlägt vor, Mensch genehmigt. Offen bleibt das Privacy-Aspektmodell selbst (wo die Zweck-/Betroffenen-Deklarationen leben).
10. **Defaults für Reorder-Fenster und Puffergrößen** pro Profil.
11. **Kanonische-Seite-Regel für Nicht-Containment-Bidi-Paare** (§5.4c) — „niedrigere stabile featureId“ bestätigen oder ein explizites Flag im Tracking-Aspekt wählen.
12. **Default-Policy für hängende Referenzen** bei aggregatübergreifenden Referenzen (§5.4g) — `KEEP` vs. `UNSET` als Default; Override pro Referenz im Tracking-Aspekt.
13. **Element-wise-Arrays in v1?** `ATOMIC` deckt die Sensorfälle ab; `SET_AT` / `RESHAPE`

(§5.4i) könnten auf eine spätere Phase verschoben werden — aber das `address`-Feld und die Delta-Arten müssen *jetzt* im Diff-Ecore reserviert werden (Disziplin der additiven Evolution, §12.3).

14. **Map-Erkennung** — wie der Tracking-Aspekt EMap-Features (Ecore-`Map$Entry`-Muster) automatisch identifiziert vs. explizite `shape=MAP`-Deklaration.

18. Phasierung & v1-Scope

18.1 v1-Scope (entschieden 18.07.2026)

Das Konzept ist eine Plattform; das größte Projektrisiko ist der Versuch, alles auf einmal zu bauen. v1 ist hart geschnitten — alles Weggelassene bleibt **in den Modellen reserviert** (die Disziplin der additiven Evolution §12.3 garantiert, dass Nachrüsten das Wire-Format nie bricht).

In v1:

1. **Query-SPI** + JPA-/Mongo-Bindings (sauber separierbar, unmittelbarer Nutzen; typisierte Comparator-Werte über ConverterService; Capability-Modell).
2. **Patch-/Stream-Metamodell** + Apply-/Invert-Engine + Konfliktprüfung (reines Modell + Bibliothek, ohne jedes Backend testbar).
3. **Capture-Hybrid** (SnapshotCapture keyed Diff; RecordingCapture mit Commit-Grenze) + CHANGELOG-Storage-Profil; Tracking-Aspekt mit den aus sensinact-mapping gehobenen Regeln.
4. **TIMESERIES-Profil** mit nur `ATOMIC`-Arrays, + Retention/Housekeeping (spaltenorientierte/Mongo-Time-Series-Layouts).
5. **Fingerprint + Snapshot-/Composite-Kette** (kanonische Form, Merkle, content-addressed Store, Upcasting on read); Genesis-Snapshots für bestehende Deployments.
6. **Manifeste**; Fault-Policies **nur** `RESYNC` + `SKIP_AND_MARK`.
7. **Ein Transport-Binding**: CloudEvents über MQTT *oder* RabbitMQ — nicht beides.
8. **Metadaten-Projektionen** (Index-Holder, abgeleitete Versionen) — begleitend, ab dem Moment, in dem der zweite Aspekt (Tracking-Config) existiert.

Explizit nicht in v1 (reserviert, dokumentiert, nicht gebaut): `ELEMENT_WISE`-Arrays (`SET_AT` / `RESHAPE`), selektives Undo mitten aus der Historie (nur Batch-Level-Undo), `BUFFER_AND_WAIT` / `QUARANTINE`-Fault-Policies, Continuous Aggregates, der EMF-Compare→ChangeSet-Offline-Konverter, der Privacy-Advisor, das zweite Transport-Binding.

Permanente Einschränkungen, keine v1-Schnitte (§22 Entscheidung #6): Single-Master-Streams; zentralisierte Modell-Governance (Fast-forward-only-Ketten).

Jede Phase ist eigenständig nützlich; nichts Späteres arbeitet Früheres um (die Koordinaten und Prinzipien werden genau deshalb jetzt fixiert, damit das gilt).

19. Technologie-Mapping (Deployment-Sicht)

Wie das Konzept auf dem bestehenden Stack landet — MongoDB, PostgreSQL, Lucene, RabbitMQ, MQTT, CloudEvents — plus die hauseigenen Bausteine Model Atlas (`fennec-model.atlas`), Apicurio- oder file-backed, auf `emf.osgi`), DDSR (`kloster-prototype`) und der Metadaten-Service (`emf.model.metadata`). Leitregel: jede Komponente unten implementiert eine *Rolle* aus §2.1; der Rollenvertrag bleibt technologie-neutral, sodass jede Zeile pro Deployment austauschbar ist.

19.1 Komponente → Technologie

Konzeptkomponente	Primärer Fit	Anmerkungen / Alternative
Ist-Zustand (Projektion)	MongoDB / PostgreSQL über die bestehenden Backends	unverändert — die heutige Persistenz <i>ist</i> diese Projektion
CHANGELOG-Stream	PostgreSQL-Append-only-Tabelle (<code>(stream_id, seq)</code> PK, Deltas als JSONB, partitioniert) oder MongoDB-Collection mit Unique-Index (<code>(streamId, sequence)</code>)	das klassische Event-Store-auf-RDBMS-Muster; einen kanonischen Log-Store pro Deployment wählen, das Log nie über zwei spiegeln
TIMESERIES-Stream	MongoDB Time Series Collections ; auf PostgreSQL: TimescaleDB	Timescale mappt 1:1: Hypertables = Serienspeicher, Retention-Policies = <code>DeletionRule</code> , Continuous Aggregates = unsere Projektionen (§15), native Kompression = das Fingerprint-Spalten-Argument (§6.3)
Snapshot-Kette / Fingerprint-Registry	Model Atlas	siehe §19.2 (Hinweis: das frühere Apicurio-Backend war kundenspezifisch und wurde entfernt)
Aspekt-/Metadaten-Registry (API)	Metadaten-Service (<code>emf.model.metadata</code>) als OSGi-Service, Artefakte gespeichert über Model Atlas	Schlüssel (<code>nsURI</code> , <code>EClass</code> , <code>Feature</code>) (§3.2)
Index-Holder / Such-Projektion	Lucene (Model Atlas bringt <code>management.lucene</code> bereits mit)	„rebuildable by design“ ist genau Lucenes Vertrag — P1 verkörpert
Projektions-Speisung (async)	MongoDB Change Streams / PostgreSQL <code>LISTEN/NOTIFY</code> oder logische Replikation	treibt Index-/Aggregat-Updates ohne Polling
Transport-Umschlag	CloudEvents	siehe §19.3 — <code>dataschema</code> trägt die Fingerprint-Auflösungs-URI

Konzeptkomponente	Primärer Fit	Anmerkungen / Alternative
Ingest (Süd)	MQTT (QoS 1)	TTN ist ohnehin MQTT; At-least-once + idempotentes Apply (§11.1) passen; Retained Messages ≈ ein Mini-get-full pro Topic
Inter-Service-Backbone (Nord)	RabbitMQ Streams (keine klassischen Queues)	Streams bieten serverseitiges Replay mit Offsets ≈ unsere Sequences; klassische Queues zerstören Historie — nur für Arbeitsverteilung nutzen, nie als das Log
Replikation / Stream-Discovery, get-full-RPC	DDSR -Channel-Modell	Prototyp-Status — siehe §19.4
Laufzeitsubstrat	emf.osgi	alle Services aus §2.1 sind DS-Komponenten; EPackages/ResourceSets als Services; Model Atlas baut bereits darauf

19.2 Model Atlas als Snapshot-Kette

Model Atlas ist das natürliche Zuhause der Snapshot-Registry: er verwaltet bereits EMF-Modelle zur Laufzeit hinter einer REST-API, mit steckbaren Storage-Backends (file-basiert, Lucene-indiziert) auf `emf.osgi`. (Ein Apicurio-Registry-Backend existierte als kundenspezifische Integration und wurde entfernt — das Schema-Registry-*Muster*, das es verkörperte, content-addressed Artefakte + Versionen + Fingerprint-Auflösung, bleibt die konzeptionelle Referenz, vgl. Kafka Schema Registry in §6.2; es ist jetzt an uns, es bereitzustellen.)

Arbeitsteilung:

- **FingerprintService** (neu, in oder neben Model Atlas): kanonische Form und Fingerprint-Berechnung — semantische Kanonisierung von Ecore + log-relevanten Aspekten, Merkle-Struktur (§6.2/§6.4). Ein Byte-Level-Hash einer serialisierten Datei ist *nicht* ausreichend; die Kanonisierung ist der wesentliche Teil.
- **Model-Atlas-Storage-SPI**: content-addressed, unveränderliche Snapshot-Speicherung (unter dem Fingerprint speichern, nie mutieren, nie GC solange referenziert — §6.4). Das ist eine hinzuzufügende Storage-Backend-Anforderung, kein Gegebenes; die bestehenden File-/Lucene-Backends sind der Startpunkt.
- **Kettensemantik** (Vorgänger-Fingerprint in jeden Snapshot eingebettet) wird vom FingerprintService beim Schreiben validiert; menschenlesbare Release-Labels sind Tags auf Kettenpositionen (§6.4) und passen zu Model Atlas' Modellmanagement-Workflow.
- Die file-backed Variante hält kleine/Edge-Deployments abhängigkeitsfrei — dieselbe Registry-API, keine Zusatzinfrastruktur (passend zur In-Band-Transportgarantie, §12.2).

19.3 CloudEvents als Wire-Umschlag

CloudEvents ist transportagnostisch (MQTT-, AMQP/RabbitMQ-, HTTP-, Kafka-Bindings) — ein Umschlag über jeden Hop, was §12.1 wörtlich ist. Attribut-Mapping:

CloudEvents-Attribut	Konzeptkoordinate
<code>id</code>	ChangeSet-ID (Idempotenz-Unterstützung)
<code>source</code>	<code>streamId</code> (Aggregat-Stream-URI)
<code>type</code>	Diff-Ecore-Nachrichtentyp (stabil, eingefroren)
<code>time</code>	Commit Time
<code>dataschema</code>	URI in den Model Atlas, die den <code>contextFingerprint</code> auflöst
<code>sequence</code> (Extension)	Pro-Stream- <code>sequence</code> — die offizielle CE-Sequence-Extension existiert
<code>data</code>	der ChangeSet-Payload (emf.codec-codiert)

Dass `dataschema` auf die Model-Atlas-Artefakt-URL zeigt, macht die Fingerprint-Auflösung zum Teil des Standardumschlags — Empfänger, die das Modell nie gesehen haben, ziehen den Snapshot von dort (Store-and-forward gilt weiterhin: Auflösung ist nur beim Apply nötig, §12.1).

19.4 Ehrliche Anmerkungen

- **Kafka ist der Lehrbuch-Fit** für die Stream-Schicht (Log, Offsets, Compaction $\approx$ Keyframes, Schema Registry), ist aber *nicht* im Stack. Das ist in Ordnung: RabbitMQ Streams + das DB-basierte Log decken dieselben Rollen zu geringeren Betriebskosten ab; Keyframes wurden in-log entworfen (§10), also wird vom Broker kein Compaction-Feature verlangt. Kommt Kafka je dazu, gleitet es ohne Konzeptänderungen in die Backbone-Zeile.
 - **DDSR ist ein Prototyp** (sprachübergreifende Service-Registry, EMF-Wire, Request/Response- + Stream-Channels mit Capability/Requirement-Matching). Sein Channel-Modell passt exakt zum Replikationsprotokoll — Diff-Subscription = Stream-Channel, get-full/Resync = Request/Response-Channel, „wer bietet welche Streams“ = Capability-Matching. Als *Kandidat* für Stream-Discovery und die Wire-Channel-Abstraktion behandeln, noch nicht als Produktionsabhängigkeit.
 - **MQTT gibt keine Ordnungsgarantie über Reconnects hinweg** — das ist kein Problem, sondern eine Bestätigung: Sequence + Fault-Policies (§11) wurden für genau diese Transportrealität entworfen.
 - **Zwei log-fähige Datenbanken** (Mongo, PostgreSQL) bedeuten Wahl pro Deployment, nicht Redundanz: das Log-Format (Diff-Ecore) ist die Portabilitätsschicht; den Log-Store zu migrieren ist „das Log replaysen“, kein ETL-Projekt.
-

20. Metamodell-Entwürfe (Ecore)

Zwei Entwurfs-`.ecore`-Dateien liegen neben diesem Dokument in `model/` —

Diskussionsartefakte, noch nicht in einem Modellprojekt (endgültiges Zuhause: offene Frage §17.8; noch kein Genmodel/Codegen):

- `model/fennec-stream.ecore` — das **Diff-Ecore** (Paket `stream`, nsURI `https://org.eclipse/fennec/stream/1.0.0`)
- `model/fennec-tracking.ecore` — der **Tracking-Aspekt** (Paket `tracking`, nsURI `https://org.eclipse/fennec/tracking/1.0.0`)

20.1 Paket `stream` — das Wire-/Log-Format

```
ChangeSet                                     (ein Batch = ein Commit, §5.1)
  id : String (id, UUID)                      streamId : String
  sequence : long                             timestamp : long (Event Time)
  commitTime : long                           contextFingerprint : String
  author, cause : String                      transactionId : String (aggregatübergreifender Link)
  manifest : boolean = false                  (Stream-Manifest: KEYFRAMEs ALLER lebenden Objekte,
§10)
  entries : ChangeEntry[*] (Containment, geordnet)

ChangeEntry                                   (invertierbares Delta, §5.2)
  objectId : String                           classId : int (CREATE; über Fingerprint)
  kind : DeltaKind                           featureId : int (-1 für CREATE/DELETE/KEYFRAME)
  index, toIndex : int = -1                   key : String                coords : int[*] ← Adresse
(§5.4)
  valueOld, valueNew : String                 state : SlotValue[*] (Containment, nur KEYFRAME)

SlotValue                                    (ein adressierter Keyframe-Slot, §10)
  featureId : int   index : int = -1   key : String   coords : int[*]   value : String

ContextSnapshot                             (Kettenglied pro Paket = der Modell-Stream, §6.4)
  fingerprint : String (id)                 predecessorFingerprint : String (fehlt = Genesis)
  created : long                             author : String
  labels : String[*] (Tags)                  packageUri : String (Kette ist pro EPackage)
  content : EObject[*] (Containment — Ecore + log-relevante Aspekte)

CompositeSnapshot                           (Deployment-Root — was ChangeSets stempeln, §6.4)
  fingerprint : String (id)                 predecessorFingerprint : String (fehlt = Genesis)
  created : long                             packages : PackageFingerprint[*] (Containment)

PackageFingerprint
  packageUri : String                       packageFingerprint : String (→ ContextSnapshot)

enum DeltaKind { CREATE, SET, UNSET, ADD, REMOVE, MOVE, PUT, REMOVE_KEY,
                SET_AT, RESHAPE, DELETE, KEYFRAME, TOUCH, MIGRATE_OUT, MIGRATE_IN }
```

Beim Modellieren getroffene Designentscheidungen:

1. **Keine Typ-Flags auf Werten.** `valueOld`/`valueNew`/`value` sind codierte Strings

(Literele oder Objekt-IDs); ob ein Wert eine Referenz, ein double, °C oder °F ist, löst sich *ausschließlich* aus `(contextFingerprint, featureId)` auf. Jedes Typ-Flag im Wire-Modell würde den Kontext duplizieren und könnte ihm widersprechen — das langweilige Modell ist das richtige (§12.3).

2. **RESHAPE**-Dimensionen reisen in `valueOld`/`valueNew` als codierte Int-Vektoren — keine Extrafelder für eine einzelne Delta-Art.
3. **Keyframes sind flache Slot-Listen** (`SlotValue`) und verwenden dieselben Adressierungsmodi wie Einträge — keine parallele Zustandsrepräsentation, die konsistent zu halten wäre.
4. **ContextSnapshot sitzt im Stream-Paket**, weil er in-band im selben Umschlag reist (§12.2). Merkle-Interna bleiben draußen — sie sind FingerprintService-Implementierung, nicht Wire-Format.
5. **Die Disziplin der additiven Evolution steht in der Paketdokumentation** — das eine Schema, das sich nicht selbst fingerprint-bootstrappen kann (§12.3).

20.2 Paket `tracking` — der Aspekt

```
TrackingRegistry                                (Root; Regeln einmal definiert, referenziert –
sensinact-Muster)
  packages : PackageTracking[*]                changeRules : ChangeRule[*]
  deletionRules : DeletionRule[*]             faultPolicies : FaultPolicy[*]      (alles Containment)

PackageTracking
  ePackage : →EPackage                        classes : ClassTracking[*]

ClassTracking
  eClass : →EClass                            classId : int (stabil, nie wiederverwendet)
  aggregateRoot : boolean = true              keyframe : KeyframeConfig
  audit : AuditMode = OFF                     features : FeatureTracking[*]

FeatureTracking
  feature : →EStructuralFeature               featureId : int (stabil, semantikgebunden, §4.2)
  mode : TrackingMode = NONE                  shape : FeatureShape = AUTO
  arrayMode : ArrayMode = ATOMIC              canonicalSide : Boolean (nur Bidi, §5.4c)
  childIdentity : IdentityStrategy = AUTO (nur Containment, §4.1)
  matchKeys : →EStructuralFeature[*] (NATURAL-Kindschlüssel für Ingest-Matching, §7.1)
  danglingPolicy : DanglingRefPolicy = KEEP (§5.4g)
  changeRules : →ChangeRule[*]                deletionRule : →DeletionRule
  faultPolicy : →FaultPolicy                  (Nicht-Containment-Referenzen in die Registry)

PersistenceRule (abstrakt: id, name, description)
  └─ ChangeRule (abstrakt)                    – §3.6, aus sensinact-mapping gehoben
    │ └─ AbsoluteChangeRule { delta }
    │ └─ PercentageChangeRule { percentage }
    │ └─ CountChangeRule { n }
    │ └─ TimeThrottleChangeRule { interval, intervalUnit }
  └─ DeletionRule { retention, retentionUnit, maxCount, cleanupInterval,
cleanupIntervalUnit }
    └─ FaultPolicy { gapPolicy, bufferTimeout(+Unit), reorderWindow(+Unit), stalePolicy } –
$11
```



```

KeyframeConfig { everyEntries, everyDuration, durationUnit }
$10

enums: TrackingMode { NONE, CHANGELOG, TIMESERIES }
      AuditMode { OFF, HEADER, TOUCHED_FEATURES }    - Ganzobjekt-Audit, §8
      IdentityStrategy { AUTO, NATURAL, SLOT, SYNTHETIC, EMBEDDED } - Kind-Identität,
$4.1
      FeatureShape { AUTO, SINGLE, ORDERED_LIST, UNORDERED, MAP, ARRAY }
      ArrayMode { ATOMIC, ELEMENT_WISE }             DanglingRefPolicy { KEEP, UNSET, CASCADE
}

      GapPolicy { BUFFER_AND_WAIT, RESYNC, SKIP_AND_MARK, QUARANTINE }
      StalePolicy { CONFLICT, DROP_AND_AUDIT, INSERT_BY_EVENT_TIME }
      DurationUnit { MILLIS, SECONDS, MINUTES, HOURS, DAYS }

```

Beim Modellieren getroffene Designentscheidungen:

6. **Das Tracking-Modell referenziert Ecore-Elemente direkt** (`ePackage` / `eClass` / `feature` als `EReferences` ins Ecore-Metamodell) — das etablierte eorm-/sensinact-mapping-/Metadaten-Muster, zur Ladezeit auflösbar.
7. **Regeln werden einmal definiert, vielfach referenziert** (Nicht-Containment-Referenzen in die Registry) — wörtlich aus `sensiNacts` `PersistenceRuleRegistry` übernommen; `FaultPolicy` tritt neben `ChangeRule` / `DeletionRule` als dritte wiederverwendbare Regelfamilie unter derselben `PersistenceRule`-Basis.
8. `shape = AUTO` leitet den Adressierungsmodus aus den Ecore-Flags ab (many/ordered/unique; `Map$Entry`-Erkennung ist offene Frage §17.14) — explizite Werte überschreiben, sodass der Normalfall keine Konfiguration kostet.
9. `aggregateRoot` markiert Stream-Eigentum (§5.4 Grundregel 3): Klassen, die nur als Containment-Kinder vorkommen, setzen es auf `false` und schreiben in den Stream ihrer Wurzel.
10. **Das Tracking-Modell ist selbst log-relevante Metadaten:** ID-Bindungen, Shapes und Kanonische-Seite-Entscheidungen beeinflussen das Lesen historischer Einträge — Tracking-Änderungen fließen durch die Snapshot-Kette und gehen in den Fingerprint ein (§17.3). Das steht in der Paketdokumentation, damit es in generierten Code überlebt.

Duplikate werden durch idempotentes Apply behandelt (`(streamId, sequence)`, §11.1) und tauchen daher in keinem Policy-Enum auf — es gibt nichts zu konfigurieren.

21. Glossar

Begriff	Bedeutung
Adressierungsmodus	Wie ein Delta den Slot lokalisiert, den es innerhalb eines Features ändert: keiner (einwertig), Index (geordnete Liste), der Wert selbst (ungeordnet), Key (Map), Koordinatenvektor (Array); bestimmt Kommutativität und Konfliktgranularität (§5.4)

Begriff	Bedeutung
Aggregate Root	Persistiertes Top-Level-Objekt; besitzt einen Change-Stream; Containment-Kinder leben in seinem Stream
Kanonische Seite	Die eine Seite eines bidirektionalen Referenzpaares, die im Log aufgezeichnet wird; die Gegenseite ist immer abgeleitet (§5.4c)
Capture-Quelle	Erzeuger von ChangeSets: SnapshotCapture (state-basiert, keyed Diff) oder RecordingCapture (commit-gescoper Recorder)
ChangeSet / Batch	Atomare Gruppe von ChangeEntries; ein Commit / Editor-Save / eine Ingest-Nachricht
ChangeRule	Deadband-/Throttle-Filter, der entscheidet, ob eine Änderung gespeichert wird (absolutes Δ , %, Zähler, Zeitdrossel)
Composite-Snapshot	Kettenglied auf Deployment-Ebene: die Liste der <code>(packageUri, packageFingerprint)</code> -Paare, content-addressed und vorgängerverkettet; sein Fingerprint (die Composite-Root) ist das, was ChangeSets stempeln (§6.4)
Kontext-Fingerprint	Reproduzierbarer, identifizierender, auflösbarer Wert, der den Interpretationskontext benennt, unter dem ein Eintrag geschrieben wurde
Genesis	Erster Eintrag eines Streams (Objekt- <code>CREATE</code>) oder einer Kette (initialer Modell-Snapshot)
Interpretationskontext	Alle Metadaten, deren Änderung das Lesen historischer Werte verändert (Modellstruktur, Feature-ID-Bindung, Typen, Einheiten, wertbeeinflussende Mappings)
Keyframe	Vollzustands-Eintrag in einem Stream; Replay-/Retention-/Resync-Anker (Video-I-Frame-Prinzip)
Keyed Snapshot-Diff	Flacher Pro-Feature-Vergleich zweier Zustände einer <i>bekannten</i> Objektidentität — kein Matching, $O(\#getrackte\ Features)$
Manifest (Stream-Manifest)	ChangeSet mit <code>manifest=true</code> : <code>KEYFRAME</code> s <i>jedes</i> lebenden Objekts des Aggregats; Abwesenheit = gelöscht; Retention-/Resync-Anker (§10)
Migrationspaar	<code>MIGRATE_OUT</code> / <code>MIGRATE_IN</code> -Einträge, transaktionsverknüpft über zwei Streams, die den Umzug eines Objekts zwischen Aggregaten aufzeichnen; pro Objekt, für Kinder nie impliziert (§5.4e)
Projektion	Abgeleitete, rekonstruierbare, asynchron gepflegte Sicht über Streams (Ist-Zustand, Index, abgeleitete Versionen, Aggregate)
Sequence	Pro Stream lückenloser monotoner long; maßgebliche Ordnung; Lücken-/Duplikaterkennung

Begriff	Bedeutung
Slot-Identität	Kind-Identität abgeleitet vom Ort (<code>parentId/featureId[index]</code>): deterministisch, koordinationsfrei; Ersetzen $\equiv$ Mutieren by design; nur wo der Slot Domänenidentität ist (§4.1)
Snapshot-Kette	Content-addressed Metadaten-Snapshots, jeder verlinkt seinen Vorgänger; <i>ist</i> der Modell-Stream; Versionen sind Positionen, Labels sind Tags
Storage-Profil	CHANGELOG (Delta-Batches) oder TIMESERIES (Absolut-Samples) — gleiches Stream-Konzept, anderes Layout
Stream	Geordnetes Log von Einträgen für ein Aggregat (Objekt-Stream) oder ein EPackage (Snapshot-Kette)
Tombstone	<code>DELETE</code> -Eintrag, der das Leben eines Objekts im Stream beendet
Zwei-Phasen-Apply	Anwendungsreihenfolge eines Batches: erst alle <code>CREATE</code> s, dann alle Feature-Deltas — macht Vorwärtsreferenzen und Zyklen innerhalb eines Batches legal (§5.4f)
Upcasting	Transform-on-read alter Einträge in den aktuellen Interpretationskontext (Einheitenkonvertierungen, Strukturmigrationen)

22. Entscheidungslog (Konsistenz-Review, 18.07.2026)

Befunde des adversarialen Konsistenz-Reviews, einzeln durchgegangen und entschieden; jeder ist in den normativen Text oben eingearbeitet. Diese Tabelle ist die dauerhafte Aufzeichnung (die frühere Arbeitsdatei `gaps.md` ist ausgemustert).

#	Entscheidung	Eingearbeitet in
1	Streams sind verlustfrei relativ zu den geltenden ChangeRules ; <code>valueOld</code> = letzter gespeicherter Wert (regelrelativer Abschluss); Regeln sind Teil des Interpretationskontexts (Regeländerung $\Rightarrow$ neuer Fingerprint, nächster Wert als Baseline gespeichert); kaskadierende Aktivierung mit Auto-Genesis; Regeln gaten nur den Stream — der Ist-Zustand bleibt live, <code>fold(stream) $\approx$ Ist-Zustand</code> innerhalb deklarierter Toleranz	§2 (P1), §5.1, §8.1 R1–R8 , §9
2	ChangeSets stempeln die Composite-Root (eine Merkle-Ebene über den Pro-Paket-Roots; <code>CompositeSnapshot</code> = verkettete <code>(packageUri, packageFp)</code> -Liste); Pro-Paket-Ketten bleiben die Evolutionseinheiten; classIds registry-weit eindeutig . Verworfen: Batch-Splitting nach Paket (bricht Einzelstream-Atomarität), Multi-Stempel, transientes Composite	§5.1, §6.4, §17.4, §20.1; stream.ecore, tracking.ecore

#	Entscheidung	Eingearbeitet in
3	Bidi-Paare: nur die kanonische Seite wird aufgezeichnet , die Gegenseite ist abgeleitet; Capture normalisiert (EMF feuert Notifications auf beiden Seiten, Reihenfolge mutationspfadabhängig — empirisch validiert, <code>EcoreEList.eInverseAdd</code>); aggregatübergreifende Gegenseite = Projektion, get-full bedient den Ist-Zustand; Tooling warnt	§5.4c, §7.2; tracking.ecore
4	Aggregatübergreifende Moves = Migrationspaare <code>MIGRATE_OUT / MIGRATE_IN (+ KEYFRAME)</code> , transaktionsverknüpft; ein Paar pro Objekt, Kinder nie impliziert; OUT child-first, IN parent-first; Containment-Deltas orthogonal (beliebiger Adressierungsmodus); Wurzeln dürfen absteigen, Kinder aufsteigen (aggregateRoot = Fähigkeit)	§5.2, §5.4e, §20.1; stream.ecore, tracking.ecore
5	Stream-Manifest : ChangeSet mit explizitem <code>manifest=true</code> , das KEYFRAMEs <i>jedes</i> lebenden Objekts trägt; Abwesenheit = gelöscht (Tombstone-GC sicher); Retention schneidet nur an Manifesten; Resync = letztes Manifest + Deltas	§10, §11.2/.3, §20.1; stream.ecore, tracking.ecore
6	Single-Master deklariert : der kanonische Log-Store vergibt Sequences (Producer dedupen über <code>ChangeSet.id</code> , Replikate apply-only); Modell-Governance zentralisiert : lineare Kette, Publishes nur fast-forward; Multi-Master bewusst nicht designet	§4.3, §6.4, §12.4
7	Privacy: das Framework dokumentiert und führt aus, entscheidet aber nie die Policy . Zweck + Dokumentation machen das Erheben legitim (Privacy-Aspekt); Schutzmechanik extern diktiert; Durchsetzung = separate Housekeeping-Jobs + Backend-TTL-Mechanismen; Löschung/Schwärzung/Crypto-Shredding sind Fähigkeiten, die diese Jobs ausführen; Advisor schlägt vor, Mensch genehmigt	§8, §17.9
8	Batches werden coalesced : höchstens ein Eintrag pro <code>(objectId, featureId, address)</code> , <code>valueOld / valueNew</code> = vor/nach dem Batch, Netto-Null wird nicht geschrieben; Ausnahme: Operationssequenzen auf geordneten Listen	§5.5; stream.ecore
9	Identitätsvertrag : <code>IdentityStrategy</code> = AUTO / NATURAL / SLOT / SYNTHETIC / EMBEDDED pro Containment-Feature; SLOT = Ortshistorie (Re-Parent = Ende + Anfang, nie MIGRATE); NATURAL-matchKeys erforderlich für mehrwertigen Ingest; SYNTHETIC mit dem Ist-Zustand persistiert; normative Regeln I1–I8	§4.1 I1–I8 , §5.4, §7.1, §17.1; tracking.ecore
—	v1-Scope-Schnitt — siehe §18.1	§18.1